A Comprehensive Evaluation of Training Data Selection for Supervised Deep Learning: [Experiments & Analysis]

Abstract

The success of supervised deep learning depends on the quantity and quality of labeled training data. However, as training datasets grow larger, they inevitably contain noisy or even harmful instances, increasing computational costs and hindering model performance. Existing data selection strategies typically focus on increasing efficiency by reducing training time without sacrificing accuracy or improving training effectiveness by selecting high-quality data. Although numerous data selection methods have been proposed, ranging from efficiency-focused to effectiveness-focused strategies and from model-agnostic to model-aware approaches, their performance varies significantly across datasets and tasks, with no universally superior method identified to date. This work presents the first comprehensive survey and empirical evaluation of representative data selection methods, spanning traditional deep learning and supervised fine-tuning (SFT) for large language models (LLMs). We introduce a unified open-source testbed that implements 15 methods and systematically evaluates them under various datasets, model architectures, etc. Furthermore, we extend the evaluation to SFT for LLMs, benchmarking 10 popular methods on 4 downstream tasks. Our results provide actionable insights into the strengths, and limitations of different strategies, offering valuable guidance to researchers and practitioners in selecting the appropriate data selection methods for their specific scenarios.

1 Introduction

Motivation. Supervised deep learning prevails in various domains. The success of deep learning models largely relies on the quantity and quality of *labeled* training datasets. However, as the volume of accessible data grows exponentially, there are inevitably redundant, noisy, or even harmful samples in the training set that increase computational costs and hinder model performance. This issue is particularly critical for large-scale models. As an example, to better solve certain downstream tasks, LLMs typically require substantial amounts of *high-quality*, *relevant* data to fine-tune the pre-trained foundation models in a supervised manner.

Consequently, data selection [5, 72], which aims to select clean and valuable instances from a large but noisy training set, has emerged as a crucial data preparation task to improve the effectiveness and efficiency of deep learning training. Existing data selection strategies typically target one of the two primary objectives, namely, optimizing *effectiveness* or *efficiency*. For example, existing studies [23, 101] demonstrate that training on a carefully selected subset can improve the effectiveness of the model by 2% to 20% compared to using the full training set, because these selected high-quality, relevant data instances prevent the negative impact of redundant or noisy information, thus improving the generalizability of the model. On the other hand, efficiency-focused data selection strategies can speed up training time by 3 to 5× without sacrificing accuracy [12].

A Summarization of Training Data Selection Methods. With respect to each objective, there are many strategies that can be categorized into different categories based on their methodologies, e.g., model-agnostic (i.e., selecting data purely considering training data distribution) or model-aware (i.e., leveraging the training metadata of the model to select data).

(1) Data Selection for Efficient Training aims to select a much smaller subset [of training data] that enables faster training while maintaining accuracy comparable to full-data training. Model-agnostic strategies [1, 28, 74, 90, 93] select subsets based solely on the geometric properties of the data, aiming to match the distribution of the full training set. Model-aware strategies [45, 64] select subsets by matching the gradients between the subset and the full training set during training. These approaches offer theoretical guarantees for achieving model accuracy comparable to full-data training.

(2) Data Selection for Effective Training aims to select a high-quality subset that maximizes model accuracy, typically using model-aware strategies that leverage the information produced during the training process. These strategies can be further divided into task-agnostic and task-aware approaches, depending on whether they explicitly consider the requirements or data characteristics of downstream tasks. Following the active learning paradigm, task-agnostic strategies [17, 19, 42, 99] typically select high-uncertainty data instances for model training. Task-aware strategies [39, 46, 47, 68, 71, 73, 77, 101] select the training [instances] that most likely improve the performance of the downstream task. In general, these methods measure the contribution of each training instance to the performance of the model based on different [objectives], including the validation loss [46, 101], influence function [47, 68, 71] or Shapley value [39, 73, 77].

(3) Extension to SFT for LLMs. Supervised fine-tuning (SFT), which fine-tunes a foundation model on a set of instruction–response pairs, effectively adapts LLMs to downstream tasks, while preserving general knowledge. Data selection plays a critical role in SFT [5, 75] because LLMs have a much higher demand for high-quality data. Although some of the above strategies for deep learning models can be applied, SFT has specialized requirements on data selection, such as quantifying instruction–response alignment [37, 53] to enhance instruction-following, or leveraging efficient techniques [96] to address the computational challenges posed by LLMs' size. At a high level, the existing data selection strategies for SFT can be classified into three categories:

i). *Model-agnostic & task-aware strategies* [37, 69, 98] improve performance on specific tasks by simply [selecting] data instances that are highly similar to the validation set of the tasks. The selection is typically based on keywords or semantic matching, without using any information about model performance.

ii). *Model-aware & task-aware strategies* [31, 96] quantify the contribution of each training instance to performance of the model on

domain-specific tasks using, for example, metadata produced during training. It then selects the training instances with the largest positive contribution.

iii). *Model-aware & task-agnostic [54, 103] strategies* aim to improve LLMs' general capability of following the instructions, rather than focusing on domain-specific tasks. They use uncertainty-based techniques to identify the most uncertain instances, or apply gradient-based methods to approximate the full data gradient.

However, different strategies perform differently on different datasets under different settings; and to the best of our knowledge, there is no one single winner within each category. Therefore, a *comprehensive experimental evaluation* is essential to guide users in selecting the appropriate method based on the property of their problem at hand. Unfortunately, to date, such work is yet to be done in both the data management and machine learning communities.

This work fills this gap by carefully analyzing and thoroughly evaluating representative methods from all main categories. These methods cover the top conference papers–including VLDB, KDD, ICML, NeurIPS, AAAI, CVPR, ACL, and ICLR–published in the past 10 years, as well as the widely recognized open-source libraries.

Our Contributions. This work makes the following contributions.

❶ ***(Comprehensive classification and review)*** **- We present an in-depth review on data selection strategies, spanning from supervised deep learning to SFT for LLMs, with a focus on efficiency and effectiveness (Section 2).** While [25] provides an overview of several data selection strategies for deep learning, [it is limited in scope and does not cover SFT.] To acquire a thorough understanding, we classify existing data selection strategies based on their targets and methodologies. Furthermore, we provide a systematic comparison of the data selection principles of these methods. To the best of our knowledge, this is the first work to thoroughly analyze both the efficiency and effectiveness of data selection strategies in the era of supervised deep learning.

❷ ***(Unified Testbed)*** **- We construct a testbed to implement 15 distinct data selection strategies for deep learning, applied across 12 datasets, all integrated through a unified interface (Section 3).** Existing work [44, 81] developed efficient and effective data selection strategies independently, CORDS [44] implements six strategies aimed at efficient training, whereas PyDVL [81] emphasizes data valuation to select high-quality subsets that improve model performance. However, they lack an end-to-end data selection workflow. Therefore, we develop an open-source testbed. (1) With regards to the varying optimization targets of the user, we implement both efficient and effective strategies for data selection. (2) Considering various real scenarios, we assembled datasets, which varied in size, data formats, and conditions of data distribution and also support user-defined datasets.

❸ ***(Comprehensive Evaluation)*** **- We evaluate various data selection strategies over various datasets and models, followed by a thorough analysis on the training efficiency and effectiveness (Section 4).** While [25] conducts experiments on 15 data selection strategies, only 2 datasets are included. We instead exhaustively evaluate 15 strategies over 12 datasets covering diverse task types, data modalities, and data sizes; and to examine their generalizability, we assess conduct evaluate across multiple model architectures. Moreover, we design our evaluation to cover representative train/test conditions: (1) in-distribution datasets with uniform data distributions; (2) out-of-distribution scenarios with distribution shifts between training and test sets.

❹ ***(Extension to LLMs)*** **- We analyze and evaluate data selection strategies for SFT (Section 5).** We first discuss how to extend the classical data selection strategies for deep learning to the context of SFT. We then cover methods specifically designed for SFT that consider the capacity for instruction-following. In total, we benchmark 10 strategies across 4 representative downstream tasks, providing the first comprehensive empirical comparison of their strengths and limitations in SFT for LLMs.

❺ ***(Insightful Findings)*** **- We present an overview of the findings from our evaluation to provide insights into selection strategies and guide future research on data selection. (Section 6).**

(1) Task-aware selection strategies can even outperform training with the full dataset. Among these strategies, Shapley value strategies often achieve the best performance, especially in those training set and test set that have distinct distributions.

(2) When training and test sets have similar distributions, efficient data selection strategies quickly match the performance of the full dataset, making gradient approximation methods sufficient and preferable due to their low time cost.

(3) The effectiveness of data selection strategies remains stable across different model architectures (including SFT for LLMs), enabling the use of lightweight models for efficient subset selection without sacrificing performance in subsequent training with larger models.

(4) Advanced techniques like LoRA are widely used in LLMs for acceleration. However, data selection methods (e.g., gradient-based methods) are not efficient enough during SFT because it usually involves only 2–5 training epochs. Thus, calculating gradients over the full dataset for selection often takes longer than training with 2–5 epochs on a chosen small subset.

(5) To enhance the ability of LLMs to perform domain-specific tasks, model-aware strategies outperform model-agnostic ones because they can evaluate the impact of each individual instance on LLM's validation performance during training.

(6) To improve the LLMs' capacity of following instructions in general, task-agnostic strategies are more effective than task-aware ones because they tend to select data with limited diversity, which in turn hinders the generalization performance of LLMs.

2 Overview

2.1 Problem Statement

Given a large candidate data pool $D = (x_i, y_i)_{i=1}^{N}$ with N labeled instances, the goal of data selection is to select a subset $S \subseteq D$ that optimizes a specific training objective. The objectives can be mainly divided into two categories: *efficient* training and *effective* training.

(1) Data Selection for Training Efficiency: The objective is to carefully select a small subset ($|S| = s \ll N$) of training data to accelerate the training process, while maintaining a performance comparable to training the model on the full training set. Formally, let [θ_S and θ_D] denote the model parameters trained on the selected subset S and the full dataset D, respectively. The efficiency-focused data selection problem can be formulated as:

Domain	Main Goal	Task Aware	Model Aware	Method	Family	Computational Complexity
Deep Learning	Efficient	✗	✗	K-means [28]	● Geometry	$O(NKd)$
		✗	✗	Herding [93]	● Geometry	$O(Nsd)$
		✗	✗	Kcenter Greedy[74]	● Geometry	$O(Nsd)$
		✗	✓	Craig [64]	■ Gradient	$O(N^2P + sN^2)$
		✗	✓	GradMatch [45]	■ Gradient	$O(NPs^3)$
	Effective	✗	✓	Least-CF [42]	✚ Uncertainty	$O(NF)$
		✗	✓	Mean-STD [42]	✚ Uncertainty	$O(NMF)$
		✗	✓	DFAL [17]	✚ Uncertainty	$O(NM(F+B))$
		✓	✓	Glister [46]	★ Validation	$O(sN(F+B))$
		✓	✓	DVRL [101]	★ Validation	$O(sN(F+B))$
		✓	✓	CG-influence [47]	◆ Influence	$O(NM(F+B))$
		✓	✓	Arnoldi [71]	◆ Influence	$O(NM(F+B))$
		✓	✓	TracIn [68]	◆ Influence	$O(TN(F+B))$
		✓	✓	TMC-shapley [23]	▼ Shapley	$O(NC(F+B))$
		✓	✓	CS-shapley [73]	▼ Shapley	$O(NC(F+B))$
LLMs Fine Tuning	Effective	✓	✗	DSIR[98]	● Geometry	$O(Nd)$
		✓	✗	BM25[69]	● Geometry	$O(NMd)$
		✓	✓	LESS [96]	◆ Influence	$O(NT(F+B+P_{\text{LoRA}}\cdot d)+NMTd)$
		✓	✓	SHED[31]	▼ Shapley	$O(NKd+MK(F+B))$
		✗	✓	PPL[97]	✚ Uncertainty	$O(NF)$
		✗	✓	NLL[27]	✚ Uncertainty	$O(NF)$
		✗	✓	Superfiltering[53]	✚ Uncertainty	$O(NF)$
		✗	✓	SelectIT[57]	✚ Uncertainty	$O(NMF)$
		✗	✓	RDS[82]	■ Gradient	$O(NMP)$
	Efficient	✗	✓	TAGCOS[103]	■ Gradient	$O(N(F+B+P_{\text{LoRA}}\cdot d)+NKd+Nsd)$

Table 1: Comparison of Data Selection Methods. For multi-round methods, we report the complexity per round. Notations: N = full dataset size, s = selected subset size, d = #-feature dimensions, K = #-clusters, T = #-outer iterations or checkpoints of training, M = #-internal iterations, F and B are respectively the time complexity of forward and backward passes; P = time complexity of gradient computation; P_{LoRA} = time complexity of gradient computation on LoRA. See Appendix A for details.

$$[S^* = \arg\min_{S\subseteq D,\ |S|=s} \left|\mathcal{F}(f_{\theta_D}, T) - \mathcal{F}(f_{\theta_S}, T)\right|] \tag{1}$$

where $\mathcal{F}(f_\theta, T)$ denotes the performance (e.g., accuracy or F1-score) of model f_θ evaluated on the downstream task T.

(2) Data Selection for Training Effectiveness: The objective is to select data to maximize the model's performance on downstream tasks T, typically by targeting data instances that are likely to have a strong positive effect on improving the model's accuracy. This can be formalized as:

$$[S^* = \arg\max_{S\subseteq D, |S|=s} \mathcal{F}(f_{\theta_S}, T)] \tag{2}$$

2.2 Data Selection for Training Efficiency

We aim to discuss how to select [a subset of full training data] so that training on this much smaller subset leads to a high efficiency, [while still achieving accuracy comparable to that obtained by training on the entire dataset.] Current strategies can be categorized into two groups: (1) Model-agnostic strategies and (2) Model-aware strategies.

2.2.1 Model-agnostic strategies These strategies, independent of any specific model architecture, rely on the geometric properties of the data distribution within the feature space to select data.

Geometry-based strategies (● in Table 1) aim to select a representative and diverse subset of data instances from the entire data set D, such that the selected subset S preserves the distribution of the original dataset. In this way, the subset S ensures that the model possess similar generalizability to the model trained on the full set.

Given that the geometric characteristic of the data distribution stays consistent during the model training, these strategies opt for a single-pass selection, which can be further categorized into three types. They vary in their representations and partitioning strategies over the feature space, [aiming to achieve better coverage and higher diversity] in the selected subset.

1). [K-means based strategies (K-means)] [28, 76, 90] partition the dataset D into K clusters and all instances are allocated to their nearest cluster centroid. For each cluster C, they can select $\frac{s\cdot|C|}{N}$ instances farthest from its centroid, where $|C|$ is the number of instances in cluster C. The overall time complexity is $O(NKd)$, where $K \ll N$ and d represents the dimension of features.

2). Herding strategies [13, 93] greedily select instances to minimize the squared distance between [the mean of the feature embeddings] of the selected subset μ_S and that of the full dataset μ_D in the embedding space, i.e., $\arg\min_{S\subseteq D} \|\mu_D - \mu_S\|^2$. It iteratively selects the instance that mostly reduces the difference between the mean of the selected instances and the mean of the full dataset. After

each selection, it updates the mean of the selected set. The process iterates until selecting s instances. The overall time complexity is $O(Nsd)$.

3). `Kcenter` [74] always selects the next instance furthest from the instances in the already selected set distance. The distance between an instance and the set is defined as the minimum distance between the instance and instances in the set. This process repeats until all s instances are selected. The time complexity is $O(sNd)$.

Analysis of Geometry-based strategies. [The above strategies aim to improve data coverage through different selection methods, each of which relies on specific assumptions that may limit its effectiveness in practice:] `K-means` achieves the coverage based on the assumption that these clusters reflect the true data distribution. However, its coverage is limited by clustering quality, and small clusters are easily neglected [8]. [Unlike clustering-based approaches, `Herding` promotes coverage by matching the mean of the feature embeddings of the selected subset to that of the full dataset; as a result, instances far from the mean can be missed [13].] [`Kcenter` explicitly selects data instances farthest from existing centers that other strategies may miss [2, 78]. This design, however, tends to select outliers, which can reduce the overall representativeness of the resulting subset [7].]

2.2.2 Model-aware Strategies Model-aware strategies explicitly leverage information obtained from the model training process, such as gradients, to guide the selection of data subsets [102].

Gradient Approximation Strategies (symbol: ■) select a weighted subset of data $S \subseteq D$ such that the overall gradient computed on S closely approximates that of D during model training. These strategies [12, 45, 64, 85, 100] begin with computing the gradient of the loss function w.r.t. the model parameters for x_i. They then construct a subset S and assign each instance in the subset a weight ω_i. [Specifically, if several instances produce similar gradients, a weighted representative can approximate their combined contribution. For instance, when x_1, x_2, and x_3 have similar gradients, setting $\omega_1 \approx 3$ allows $\omega_1 \nabla f(x_1)$ to approximate their summed gradients.] In this way, the subset S effectively represents the overall gradient information of the full dataset D. Formally, the objective is to minimize the gradient approximation error:

$$(S^*, \mathbf{w}^*) = \arg \min_{\substack{S \subseteq D, |S|=s \\ \mathbf{w} \in \mathbb{R}^{|S|}}} \left\| \sum_{i \in S} w_i \nabla_\theta \mathcal{L}(f_\theta(x_i)) - \sum_{j \in D} \nabla_\theta \mathcal{L}(f_\theta(x_j)) \right\|_2^2 \tag{3}$$

[where $\mathbf{w} = \{w_i\}_{i \in S}$ denotes weights assigned to the selected instances, and $\nabla_\theta \mathcal{L}(f_\theta(x_i))$ is the gradient of the loss with respect to model parameters θ for instance x_i.]

Next, we discuss two classical methods. 1) `CRAIG` [64] iteratively selects a subset during training by minimizing the upper bound of the gradient approximation error. It first computes the pairwise gradient similarities ($O(N^2P)$), then greedily selects s instances by sampling K candidates per step and chooses the one that reduces the error most, with a complexity of $O(N^2P+sKN)$. 2) `GRAD-MATCH` [45] directly solves [Equation (3)] through orthogonal matching pursuit (OMP) [67]. Thus, it allows non-negative real-valued weights, while `CRAIG` only supports integer weights. With a time complexity of $O(NP + NPs^3)$, it achieves finer gradient matching and higher accuracy than `CRAIG` [at the expense of increased computational cost.]

Analysis of Gradient Approximation Strategies. *(1) Complexity analysis:* for gradient approximation strategies, the complexity depends on how gradients are computed and aggregated. Some strategies [12] utilize batch-level approximation [and] stochastic sampling, which significantly reduces the complexity compared to `CRAIG`. Overall, geometry-based methods are typically much more efficient than gradient-based ones, as they avoid repeatedly computing gradients during training. *(2) Effectiveness analysis:* For gradient approximation strategies, performance largely depends on [how closely] the weighted subset gradient approximates the full gradient. Methods that more closely approximate the full gradient, such as Grad-Match, generally achieve better selection performance than methods like CRAIG that only minimize an upper bound of the approximation error. By explicitly leveraging gradient information, these strategies typically outperform geometry-based strategies, which only consider data distribution and neglect model-related information produced during training.

2.3 Data Selection for Training Effectiveness

Data selection for effective training aims to select a high-quality training subset from D to maximize the model accuracy. Hence, naturally, all popular strategies are model-aware, i.e., [leveraging] the model prediction results or metadata during training to select data. [Building on this,] we further categorize these strategies into task-agnostic and task-aware, depending on whether or not they [explicitly account for] downstream tasks because the distribution of training data may differ from that of downstream tasks.

2.3.1 Task-agnostic Strategies Task-agnostic strategies refer to data selection strategies that do not leverage feedback from downstream tasks, such as accuracy on a validation set. Instead, they mainly rely on the prediction uncertainty of training instances to score and select data [24].

Uncertainty-based Strategies (symbol: ➕) are often combined with the active learning paradigm, *i.e.*, iteratively selecting data by identifying instances where the model is the most uncertain or least confident about their prediction results. These instances are expected to have the greatest impact on model performance. In practice, these strategies start with an empty training set and iteratively compute an uncertainty score for each instance. At each step, the top-K instances with the highest uncertainty are added to the training set until reaching the budget s.

(1) Single Forward Pass Uncertainty [42] computes the uncertainty for each data instance based on a single forward pass of a model [21]. These strategies differ in the specific metrics used to quantify uncertainty, such as entropy [32], least confidence [52], or the margin between the top 2 predicted classes [42]. The computational complexity is $O(NF)$, where F is the cost of a single forward pass of the model.

(2) Decision Boundary Uncertainty [17, 99] estimates uncertainty by estimating how close an instance is to the model's decision boundary. Given an instance x_0, the distance of x_0 to the decision boundary is calculated by iteratively perturbing x_0 along the gradient direction of the model output (i.e., $x_{i+1} = x_i + \alpha \nabla_x \mathcal{L}(f_\theta(x_i))$, where $f_\theta(x)$ is the model output and α is the step size). This procedure has an overall complexity of $O(NMB)$, [where M denotes the

number of perturbation steps,] N the number of instances, and B the cost of one backward pass.

(3) Ensemble-based Uncertainty [19, 42, 43, 66] estimates uncertainty by aggregating predictions from multiple stochastic forward passes. Myers et al. [66] use the variation ratio to measure the uncertainty as $U(x) = 1 - \frac{T}{M}$, where T is the frequency of the class most predicted among M passes. Higher values indicate higher uncertainty. The computational complexity is $O(NMF)$.

Analysis of Uncertainty-based Strategies. Uncertainty-based strategies differ in both computational complexity and effectiveness, depending on their underlying calculation strategy. *(1) Complexity analysis:* single forward pass uncertainty strategies are computationally efficient because they only need a single pass. Decision boundary based strategies are slower because they require multiple gradient-based updates per instance. Moreover, ensemble-based strategies that use multiple stochastic forward passes incur higher computational costs due to repeated model evaluations. *(2) Effectiveness analysis:* targeting hard-to-classify instances, margin-based and decision boundary uncertainty methods enhance model performance and therefore often outperform the single forward pass strategies. [Ensemble-based uncertainty aggregates predictions from multiple independently trained models, thereby reflecting variability across model instances and yielding more reliable uncertainty estimates] [18, 22].

2.3.2 Task-aware Strategies Task-aware strategies leverage explicit feedback from downstream tasks, typically using a validation set drawn from the target task to guide data selection [94]. By directly estimating how adding or removing specific data instances affects the performance on the validation set, these strategies are able to select the training data expected to yield the largest improvement w.r.t. the target task. They can be further categorized into three categories based on how they quantify the impact of each data instance on the performance of downstream tasks.

❶ Validation Loss-based Strategies (Symbol: ★) iteratively select data instances from the full dataset that most effectively reduce the model's validation loss. At each step, the model is updated with the currently selected subset, and the candidate instances are then evaluated using the updated model to identify those most likely to yield further reductions in validation loss. For instance, `Glister` [46] formulates the data selection problem as a bi-level optimization that selects multiple subsets iteratively and minimizes the training loss on each subset. Besides, `DVRL` [101] models the data selection process as a reinforcement learning (RL) problem, where a value estimator utilizes validation loss as a reward to learn the selection probability of each data instance. Similarly, recent approaches [11, 56, 91] [formulate data selection as a multi-armed bandit problem [83], where data instances are selected based on validation loss as a reward signal, while the exploration–exploitation mechanism implicitly encourages data diversity.] RL-based strategies generally incur a time complexity of $O(sN(F+B))$ per training iteration.

Analysis of Validation Loss based Strategies. *(1) Complexity analysis:* Bi-level optimization and RL based strategies incur high computational cost as dataset size grows because they evaluate too many candidate subsets. Therefore, compared to other strategies, they are typically not efficient enough, especially [when processing] large-scale datasets. *(2) Effectiveness analysis:* Bi-level optimization strategies greedily select subsets to minimize validation loss, and `DVRL` develops a policy to select data by maximizing the cumulative reward from validation performance gains. [By explicitly leveraging validation signals that reflect downstream task objectives, such validation-driven strategies are generally more effective than task-agnostic approaches, which rely solely on training data statistics.]

❷ Influence function Strategies (symbol: ◆) estimate the impact of *each training instance* on model performance using a validation set and select those with the highest influence scores for subsequent training. The core idea is to evaluate how removing or reweighting a single training instance would affect the validation loss and then prioritize the instances with the most beneficial impact. This approach is inspired by the classical leave-one-out (LOO) retraining algorithm [47], which measures the impact of removing or reweighting each training instance. Specifically, given an instance x_i, the LOO influence is defined as: $\mathrm{LOO}(x_i) = \mathcal{L}(f_\theta(D)) - \mathcal{L}(f_\theta(D \setminus x_i))$, where $\mathcal{L}(\cdot)$ denotes the empirical loss. However, as the dataset size increases, LOO becomes computationally prohibitive. [Influence functions provide an efficient first-order estimate of the leave-one-out (LOO) effect without explicit retraining. Rather than physically removing a training instance, they assess its contribution through an infinitesimal downweighting perturbation, enabling efficient evaluation of each instance's effect on validation performance.]

Formally, the influence of a training instance x_i on the loss of a validation set D_{val} can be quantified as [47]:

$$[\mathcal{I}(x_i) = \sum_{x_j \in D_{val}} -\nabla_\theta \mathcal{L}(f_\theta(x_j))^\top H_\theta^{-1} \nabla_\theta \mathcal{L}(f_\theta(x_i))] \tag{4}$$

where $H_\theta = \nabla_\theta^2 \mathcal{L}(f_\theta(\cdot))$ is the Hessian matrix with respect to the model parameters θ.

Analysis of Influence Function Strategies. *(1) Complexity analysis:* while influence function provides a first-order approximation, [their computational cost], particularly the need to invert the Hessian matrix, can be very expensive for large models and datasets, with complexity $O(P^3 + NP^2)$. [Here, the P^3 complexity arises from computing the inverse Hessian H_θ^{-1}, while the NP^2 complexity corresponds to computing influence scores for all N training instances. As a result, influence function strategies typically rely on approximate optimization techniques to reduce the overall computational cost.] `CG-influence` [47] and `Arnoldi` [71] simplify H_θ^{-1} using iterative linear equations. `TracIn` [68] estimates the impact by summing the dot products of gradients between training and validation instances over several checkpoints. Influence function strategies are typically less efficient than geometry-based, gradient approximation and uncertainty-based strategies because they require iterative gradient computations, involving full forward and backward passes over the training data, which is time-consuming. *(2) Effectiveness analysis:* `CG-influence` and `Arnoldi` generally outperform `TracIn`, as they compute the standard influence function scores involving H_θ^{-1}, whereas `TracIn` omits this term and therefore provides a less precise approximation. Compared to validation loss-based strategies, influence function strategies offer finer granularity by assessing the impact of each instance on model parameters, while validation loss-based strategies only show the combined effect

of all instances. Recent studies [41, 47, 68] also show that influence-based approaches outperform [validation loss-based] ones in data selection.

❸ **Shapley value strategies** (symbol: ▼) evaluate the importance of each training instance by computing the average improvement of model performance on a validation set when a training instance is added to any possible subsets of the training dataset [23, 39, 73]. Instances with higher Shapley Values are considered more prioritized for selection. Formally, the Shapley Value (SV) of an instance x_i ($x_i \in D$) is given by:

$$\mathcal{I}_{SV}(x_i) = \sum_{S \subseteq D \setminus \{x_i\}} \frac{v(S \cup \{x_i\}) - v(S)}{\binom{N-1}{|S|}} \tag{5}$$

where $v(S)$ is the utility function, which represents the model performance (e.g., accuracy or negative loss) on the validation set after training with the subset S.

Analysis of Shapley Value strategies. *(1) Complexity analysis:* Exactly computing SV is impractical for large datasets, as it requires evaluating all $[2^{N-1}]$ possible subsets. Therefore, most existing works [23, 38, 39, 61, 73, 95] employ approximation strategies to estimate it efficiently. Common approximation methods include Monte Carlo sampling (e.g., `TMC-shapley` [23],`CMC-shapley` [55], `CS-shapley` [73]), which randomly samples $[C]$ dataset permutations instead of all possible subsets and computes the average improvements along each permutation, reducing the complexity to $[O(CN(F+B))]$. Overall, SV strategies are [computationally more expensive] than other strategies. *(2) Effectiveness analysis:* SV strategies are supposed to perform better than influence function in theory [79, 87]. The reason is that the utility function v in SV can represent any metric specified by the user. If [utility function] is the validation loss, i.e., $v(x) = \mathcal{L}(f_\theta(x))$, SV is equivalent to averaging the influence function of each instance across all possible subsets. However, standard influence functions consider only the difference between [the full training dataset and the corresponding leave-one-out training set.]

2.4 Data Selection for SFT

Inspired by the above data selection techniques in deep learning, [a series of] strategies have been proposed to enhance the effectiveness of LLMs during the SFT stage. One of the main goals of SFT is to enhance the capabilities of LLMs on domain-specific tasks [96]. Therefore, the strategies in this scenario are all task-aware, which can be categorized into model-agnostic and model-aware strategies. The other goal is to improve the [instruction following capacity of LLM] for general tasks [53]. Therefore, task-agnostic strategies [work well]. Note that in the SFT context, there are no model-agnostic & task-agnostic strategies, because such strategies (e.g., deduplication) are typically applied during the data cleaning stage of pre-training [50]. In summary, there are 3 categories of data selection strategies for LLM SFT: *model-agnostic & task-aware* strategies, *model-aware & task-aware* strategies, and *model-aware & task-agnostic* strategies.

2.4.1 Model-agnostic & task-aware strategies These strategies select data instances with a distribution similar to the validation set using keyword-based or semantic similarity measures.

❶ **Geometry-based Methods**: Instead of selecting a subset that represents the distribution of the whole dataset as discussed in Sec. 2.2, geometry-based methods in SFT aim to select a subset that represents the distribution of the validation set. Specifically, recent works (e.g., `DSIR` [98], `BM25` [69], `RDS+` [37]) propose task-aware adaptations that combine geometric selection with task-specific information for SFT, such as the similarity of training instances to the target test data in feature space. Specifically, `DSIR` hashes the full dataset and validation set into n-gram feature vectors, and estimates their hashed n-gram distributions p_D and p_{val} to capture frequency patterns in each set, in $O(Nd)$ time complexity. Importantly, these methods use condensed feature spaces rather than storing each individual validation instance, and thereby avoid overfitting [98].

2.4.2 Model-aware & task-aware strategies. These strategies select data instances by measuring the contribution of each training instance to LLM's performance (e.g., `Mistral-7B`) for domain-specific tasks (e.g., `GSM8K`).

❶ **Influence Function based Methods:** In SFT, applying classical influence functions is infeasible due to the prohibitive cost of storing and inverting the Hessian matrix for models with billions of parameters. To overcome these limitations, recent works [88, 96] employ `TracIn` [68] in deep learning, which directly computes the gradient products between training and validation instances, [and avoids] Hessian matrix computations. To further reduce memory usage during gradient calculation, `LESS` [96] incorporates the LoRA [33] technique and random projection [9].

❷ **Shapley Value based Methods:** The main bottleneck of Shapley value estimation is repeated model retraining on multiple subsets, which is impractical for LLMs [87]. To address this, recent approaches [31, 72, 87] propose various methods to accelerate Shapley value estimation. For instance, `SHED` [31] employs a two-stage approximation: it first clusters the data instances and then computes Shapley values only for representative proxy instances (cluster centers). It then assigns these values to all members of the corresponding clusters. [This approach improves efficiency by computing Shapley values only for a representative subset of training instances.]

2.4.3 Model-aware & task-agnostic strategies. [These strategies select training instances based solely on the target model (e.g., `Mistral-7B`), without relying on downstream tasks, and evaluating the model across multiple downstream tasks, aiming to enhance the general instruction-following capabilities of LLMs.]

❶ **Uncertainty-based Methods:** In SFT, sentence-level uncertainty is often used to guide data selection, such as negative log-likelihood (NNL) [27], perplexity (PPL) [97] and instruction-following difficulty (IFD) [53, 54]. Specifically, given an instance with instruction x and response y, NLL is quantified as $[NLL(y|x) = -\frac{1}{T}\sum_{t=1}^{T} \log p(y_j|x, y_1, \cdots, y_{t-1})]$, where T is the length of y and y_t represents the t-th token, and $[\mathrm{PPL}(y|x)]$ is defined as $\exp(\mathrm{NLL}(y|x))$. These metrics capture the accumulated uncertainty of the model when predicting each token in the response; higher scores indicate greater uncertainty. IFD is defined as $IFD(y|x) = \frac{PPL(y|x)}{PPL(y)}$, where $PPL(y|x)$ and $PPL(y)$ denote the perplexities of the given model in fitting response y with and without the instruction x, respectively. This metric uses loss entropy to identify discrepancies between the expected responses of a model and its generation capability, which is widely used in SFT data selection [29, 58].

Moreover, SelectIT [57] leverages loss entropy to obtain a set of scores for each instance, using multiple LLMs and prompts; instances with more inconsistent scores are considered to have higher uncertainty. Instag [60] uses LLMs to assign descriptive tags (e.g., task type, topic, domain) to each instance, [and explicitly defines uncertainty as the number of tags.] More tags imply greater semantic complexity and inconsistency, hence higher uncertainty.

❷ **Gradient Approximation-based Methods:** The high dimensionality of LLM gradients [leads to prohibitive computational and memory costs, making gradient-based selection impractical in practice.] Thus, TAGCOS[103] leverages LoRA [33] and random projection [9] to enable scalable gradient matching more efficiently. For each data instance, TAGCOS extracts LoRA gradients and projects them to a d-dimensional space, with a [total] cost of $O(N(F+B) + NP_{\text{LoRA}}d)$. Then it clusters the projected gradients using K-means [with a cost of $O(NKd)$,] and selects $\frac{s}{K}$ representatives within each cluster [with a cost of $O(sNd)$]. Thus, the total complexity is $O(N(F+B) + NP_{\text{LoRA}}d + NKd + sNd)$.

3 Experimental testbed

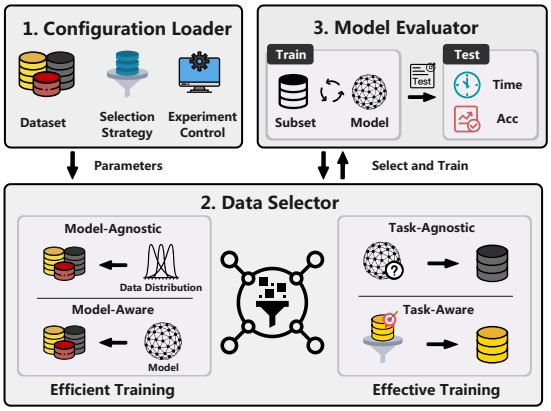


Figure 1: Workflow of the Proposed Testbed.

We develop the first open-source testbed for data selection that (1) implements 15 representative strategies, spanning model-agnostic/model-aware and task-agnostic/task-aware strategies of efficient and effective data selection; and (2) conducts evaluation on 12 datasets with various data distributions and in rich scenarios with various user-specified parameters. Furthermore, our testbed is flexible in that it allows users to easily apply the featured data selection strategies to user-provided datasets and tasks.

As shown in Figure 1, the testbed includes three modules: *(1) Configuration Loader* allows users to configure datasets, models, parameters, data selection strategies, and experimental controls (e.g., selection fraction). *(2) Data Selector* offers 15 data selection strategies for efficient and effective training. Specifically, it runs a data selection strategy specified in the user configuration to select a subset of data and passes it to the Model Evaluator. For iterative strategies, the Data Selector and Model Evaluator interact to continuously improve the model performance. *(3) Model Evaluator* trains the target model using the selected data subset and evaluates its performance on the test set, measuring both accuracy and latency.

Datasets. Our testbed uses 12 public datasets covering diverse domains and modalities, with varying scales and distributions. In the *in-distribution scenario* (ID), where the training and test sets share the same data distribution, we evaluate 8 datasets at various scales, all collected from real-world scenarios. (1) CIFAR-100 [48] and (2) Tiny-ImageNet [70] are large-scale image datasets characterized by rich visual diversity. (3) Adult [3], (4) IMDB-Large [86], (5) CoverType [10], and (6) Bike [62] are tabular datasets with diverse characteristics: Adult is a widely used census income prediction dataset, IMDB-Large contains more than 20 million records, CoverType is a large-scale multiclass classification dataset, and Bike is used as a regression benchmark. (7) IMDB [51] is a binary text classification task and (8) News [65] is a multiclass text classification task. The training and test sets for the mentioned datasets exhibit identical data distributions, and the proportion of train, validation and test set is 80%, 10% and 10%.

Category	Dataset	Type	#Size	Task
ID	CIFAR-100	Image	60,000	classification
	Tiny-ImageNet	Image	100,000	classification
	Bike	Tabular	40,000	regression
	Adult	Tabular	48,842	classification
	CoverType	Tabular	581,012	classification
	IMDB-Large	Tabular	21,303,410	classification
	IMDB	Text	50,000	classification
	News	Text	210,294	classification
OOD	MNIST→MNISTM	Image	59,001	classification
	MNISTM→MNIST	Image	59,001	classification
	HR	Tabular	50,000	classification
	House	Tabular	32,963	regression

Table 2: Statistics of Datasets.

However, in practical applications, the data used for training a model may differ from the data it encounters in downstream tasks during deployment and evaluation, resulting in distribution shifts between the training and test sets. To analyze this *out-of-distribution* (OOD) scenario, we select four datasets from various modalities that exhibit such distribution shifts. (1) MNIST→MNISTM [20] is an image classification task where models are trained on MNIST digits, but validated and tested on MNISTM, [a synthetic variant of MNIST with complex background patterns.] (2) In contrast, MNISTM → MNIST [20] trains a model on MNISTM, but validates and tests the model on the original MNIST dataset. (3) HR [11] is a tabular data set for the prediction of employee attrition, where training is performed on data from the Sales, International, Purchasing, Marketing, and Technology departments, while testing is performed on the Finance department. This dataset is collected from real-world organizational data, reflecting actual employee records and attrition events. (4) House [11] is a real-world tabular regression data set to predict house prices in Indian cities, where training is performed on data from Bangalore, Chennai, Delhi, Kolkata and Hyderabad, while the downstream evaluation targets Mumbai. We directly use the train, validation and test set that have already been splitted in the above works for our evaluation. In fact, they construct OOD datasets based on the method in [23] (see Appendix B.1.1 for details).

Evaluation Metric. We use *F1-score* to evaluate data selection strategies on classification tasks and *mean squared error (MSE)* on regression tasks. The *F1-score* measures a model's ability to distinguish between classes, defined as F1-score $= 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$,

where Precision and Recall are calculated based on predicted and ground truth labels. *MSE* measures the average squared difference between the normalized ground truth values (y) and the predicted values (y_{predict}), defined as MSE $= \frac{1}{n}\sum_{i=1}^{n}(y - y_{\text{predict}})^2$, where n refers to the [size] of test data. A lower MSE indicates better predictive performance. Efficiency is evaluated by data selection time and model training time.

Implementation. Our testbed is implemented in Python 3.7. Experiments are conducted on a cluster with four NVIDIA RTX 3090 GPUs (24GB each), dual Intel Xeon Gold 6148 CPUs (80 threads total), and 1 TiB RAM. We employ Optuna [4] to determine the hyperparameters (e.g., learning rate, etc.) with grid search (see Appendix-B.1.2 for details).

4 evaluation on Deep Learning

We systematically evaluate 15 data selection methods for deep learning, including efficiency-focused and effectiveness-focused strategies and highlight their strengths and limitations.

Strategies. As discussed before, we categorize data selection methods into 5 strategies: (1) Geometry-based strategies (`K-means` [28], `Herding` [93], and `Kcenter` [74]) are model-agnostic methods. (2) Gradient approximation strategies (`Craig` [64] and `GradMatch` [45]) are model-aware and select data based on the gradients. (3) Uncertainty-based strategies (`Least-CF` [84], `DFAL` [17], and `Mean-STD` [43]) are task-agnostic and iteratively select data instances with active learning. (4) Validation loss strategies, `DVRL` [101] and `Glister` [46], are both task-aware, which respectively use RL and bi-level optimization to iteratively select data. (5) Influence function strategies (`TracIn` [68], `Arnoldi` [71], and `CG-influence` [47]) and (6) Shapley Value strategies (`TMC-shapley` [23] and `CS-shapley` [73]) are all task-aware. After estimating the importance score of each data instance, they perform data selection in one round.

Experimental Setting. We test various data selection rates for all methods, i.e., 10%, 30%, 50%, and 70%. All image datasets were processed using a standard `ResNet-50` backbone pretrained on ImageNet. For tabular datasets, we employ an `MLP` with two hidden layers (128 units, ReLU) followed by an output layer. For text datasets, we first extract sequence-level embeddings using a pretrained `E5` model [89], which are subsequently fed into an `MLP` classifier with two hidden layers for text classification tasks. To investigate the impact of model architecture, we conduct experiments on the `HR` dataset using three different models: `MLP`, `TabNet` [6], and `TabTransformer` [35]. For `CIFAR-100`, we use `ResNet18`, `ResNet50`, and `Vision-Transformer` [16], allowing us to evaluate how model complexity influences the performance of different data selection methods. Due to space constraints, error bars for all reported results are included in Appendix C.1. Additional evaluation results on other datasets are also provided there. [For each setting, the experiments are performed for five times.]

Our analysis aims to address the following questions:

- **RQ 4.1**: How do various data selection strategies perform in accuracy on diverse datasets?
- **RQ 4.2**: How efficient are different strategies for selecting data computationally, and which strategies offer [the best performance?]
- **RQ 4.3**: How do different model architectures affect the effectiveness and efficiency of data selection strategies?

4.1 Overall Effectiveness Comparison

We measure F1-score of classification and MSE of regression tasks. The results are shown in Table 3. On datasets `CIFAR-100`, `Adult`, and `News`, we observe that Geometry-based strategies (🟢) perform the worst, with `Kcenter` achieving the best results among them because of diverse coverage. Gradient approximation strategies (🟥) consistently outperform geometry-based strategies, as they leverage gradient information to select a subset that represents the full training set. For example, `GradMatch` outperforms `K-means` by more than 13% in the 30% fraction of `CIFAR-100`, while `Craig` exceeds `Herding` by more than 5% in the 70% fraction of `News`, approaching the method of using the full dataset.

Key Observation 1: Gradient approximation strategies consistently outperform geometry-based strategies by approximating the full dataset gradients during model training, resulting in performance comparable to training on the full training dataset.

In addition, gradient approximation strategies also outperform other strategies for data effectiveness (blue background in Table 3) on [ID] datasets. Because training and test data share similar distributions, using the full dataset typically yields the best accuracy. The gradient approximation effectively minimizes the performance gap between the selected subset and the full dataset.

Key Observation 2: For the in-distribution scenario, efficiency-focused strategies (e.g., gradient approximation) always perform the best because the distributions of training and test dataset are similar, and they precisely approximate the distribution of training data, achieving their primary optimization goal of improving training efficiency.

However, on datasets `MNIST`→`MNISTM` and `House` with the out-of-distribution property (i.e., the training dataset differs from the test one), effectiveness-focused strategies, such as Shapley Value strategies (🔽), generally outperform geometry-based and gradient approximation based ones on F1-score and MSE. For example, `CS-shapley` outperforms geometry-based strategies by more than 5% across all fractions on `MNIST`→`MNISTM`. The reason is that geometry and gradient based strategies select data instances purely with the goal of approximating the full training set, [without incorporating the requirements of downstream tasks.]

Key Observation 3: For the out-of-distribution scenario, effectiveness-focused strategies (e.g., influence function or Shapley Value) always perform the best. This is because they fully leverage the [information of] downstream tasks to improve the model accuracy. [Therefore, they could narrow the gap between the distribution of training and test dataset, leading to more effective and stable predictive performance.]

Among effectiveness-focused strategies, we observe that task-aware strategies (e.g., validation loss, influence function and Shapley Value) outperform task-agnostic strategies (e.g., uncertainty-based) on almost all datasets because task-agnostic strategies only consider the model uncertainty without customizing to the downstream tasks. Specifically, within uncertainty-based strategies (➕),

Method	CIFAR-100 (ID)				Adult (ID)				News (ID)				MNIST→MNISTM (OOD)				House (OOD)-MSE			
	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7
Random	30.1	56.8	69.5	75.7	81.2	81.8	83.1	83.7	34.3	48.9	52.1	53.9	34.8	38.5	40.7	43.4	1.101	0.987	0.895	0.882
● K-means	37.5	66.3	69.8	77.5	82.4	84.0	84.4	84.4	27.5	43.8	50.7	52.8	36.6	39.8	42.5	44.1	0.937	0.922	0.897	0.888
● Herding	40.7	65.7	76.2	79.3	83.0	84.2	84.2	84.4	27.6	38.0	47.8	52.1	33.6	37.8	40.5	43.7	0.935	0.921	0.891	0.882
● Kcenter	51.7	76.1	78.3	80.5	81.2	83.7	84.0	84.8	36.6	48.9	51.3	53.2	35.7	41.7	43.5	43.7	0.934	0.919	0.888	0.873
■ Craig	68.3	78.9	79.9	82.6	83.6	84.3	85.1	85.5	42.2	49.7	53.2	54.5	38.6	42.5	45.1	45.6	0.884	0.873	0.866	0.862
■ GradMatch	**77.4**	**79.7**	**80.3**	**83.1**	**83.7**	**84.7**	**85.3**	**85.6**	**44.2**	**50.3**	**53.5**	**54.6**	38.8	42.8	45.5	45.6	0.883	0.872	0.864	0.862
✚ Least-CF	40.3	64.6	75.6	77.4	81.5	83.0	84.0	84.5	22.7	40.2	50.2	52.8	37.3	39.7	42.1	45.1			-	
✚ DFAL	49.6	78.0	79.5	81.1	79.7	84.0	84.1	84.5	36.6	47.3	52.1	53.4	39.4	43.2	45.1	47.3			-	
✚ Mean-STD	44.6	66.9	73.0	76.0	83.2	84.1	84.3	84.7	37.8	50.0	52.2	53.6	39.8	43.4	45.3	47.4	0.881	0.871	0.862	0.857
★ Glister	67.7	73.1	79.2	81.9	82.6	83.5	84.8	85.2	35.1	38.1	42.8	52.2	43.5	44.5	45.3	47.6	0.875	0.867	0.856	0.846
★ DVRL	64.2	68.2	77.5	81.7	83.3	84.2	84.2	84.6	38.1	48.7	52.2	53.7	43.5	44.7	45.6	48.7	0.873	0.863	0.852	0.844
◆ TracIn	63.4	72.4	79.1	81.8	83.2	83.6	84.1	85.1	37.0	49.8	52.2	54.4	43.6	45.3	45.7	48.8	0.873	0.853	0.847	0.843
◆ CG-influence	58.4	76.1	79.5	82.5	82.3	83.2	84.6	85.1	39.0	49.5	52.1	53.9	43.8	45.4	46.2	50.4	0.869	0.854	0.846	0.843
◆ Arnoldi	61.4	76.3	79.7	82.5	82.1	83.4	84.9	85.2	37.5	41.3	50.9	53.7	43.9	45.5	46.4	50.3	0.869	0.853	0.844	0.841
▼ TMC-shapley	62.4	74.6	79.8	82.3	81.7	82.7	84.2	85.0	36.9	41.3	50.2	54.2	44.1	45.6	46.8	50.8	0.868	0.843	0.836	0.821
▼ CS-shapley	63.1	76.4	79.8	82.5	83.2	83.8	85.0	85.1	36.6	48.5	51.6	54.3	**44.9**	**45.6**	**47.3**	**51.2**	**0.867**	**0.842**	**0.835**	**0.819**
Full	**83.3**				**85.6**				**56.7**				**45.6**				**0.862**			

Table 3: Performance of different data selection methods in deep learning. The bold and underlined values represent the first and second best performance, respectively. Efficient selection strategies are marked in orange, and effective ones in blue. Least-CF and DFAL are specifically designed for classification tasks thus not applicable to the regression dataset House.

ensemble-based uncertainty outperforms single forward-pass uncertainty by aggregating predictions from multiple runs, thus better capturing model uncertainty. For example, CS-shapley, CG-influence and Glister outperform Mean-STD by more than 2%, 1%, 0.5% across all fractions on MNISTM→MNIST dataset, indicating that the information gained from downstream tasks significantly enhances performance.

Key Observation 4: Task-aware strategies outperform task-agnostic ones because the former [leverage the validation set to select instances that closely align with the underlying distribution of downstream tasks.]

Among task-aware strategies, influence function (◆) and Shapley Value (▼) based strategies generally perform better than validation-loss based strategies (★). For example, CS-shapley and Arnoldi outperform Glister by around 3% on MNISTM→MNIST. This is because validation-loss based strategies are based solely on the overall loss of validation set to guide data selection, while the latter two strategies consider the impact of each instance on the downstream tasks. Furthermore, Shapley Value strategies outperform influence function methods, as influence functions only consider the impact of removing a single instance from the entire dataset, whereas Shapley Value accounts for the contribution of each instance across all possible subsets.

Key Observation 5: Influence function and Shapley Value-based methods outperform validation-loss-based methods, as the former provides a more fine-grained evaluation by quantifying the individual impact of each training instance on the validation set, leading to more effective data selection.

4.2 Overall Efficiency Comparison

In this section, we report the end-to-end computation time including data selection and model training. The results are presented in Table 4. Due to space limitation, the breakdown of data selection time and training time is provided in Appendix C.1.

As shown in Table 4, both Random and geometry-based strategies are highly efficient, but they do not perform well on model accuracy. However, gradient approximation strategies (Craig and GradMatch) achieve a 2× speedup over the full-gradient method on CIFAR-100 and Adult without sacrificing accuracy.

Key Observation 6: For the in-distribution scenario, gradient approximation strategies are the best fit, as they offer a favorable balance between strong performance and high efficiency.

However, as discussed above, for the out-of-distribution scenario, influence function and Shapley Value strategies are highly effective [but the associated computational complexity is very high.] For example, on MNIST→MNISTM, CS-shapley is 3 times slower than training on the full dataset. On the other side, uncertainty-based strategies are less effective, they result in inferior performance.

Key Observation 7: Influence function and Shapley Value strategies offer the best performance but are computationally intensive, while uncertainty-based strategies are significantly more efficient but less effective. Thus, users are encouraged to select the strategy that best aligns with their specific requirements.

4.3 Different Model Architectures

In this section, we report the performance of data selection strategies using different model architectures. We report the top-performing method of each category when selecting 50% of the training data for illustration.

We observe from Figure 2 that data selection remains effective [across diverse model architectures.] However, higher model complexity leads to longer training time; thus, selecting an appropriate data subset is critical for enhancing training efficiency. In the in-distribution scenario, gradient approximation strategies still significantly improve the training efficiency. For instance, gradient

Method	CIFAR-100 (ID)				Adult (ID)				News (ID)				MNIST→MNISTM (OOD)				House (OOD)			
	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7	0.1	0.3	0.5	0.7
Random	411	1366	2253	3071	11	27	53	81	14	32	59	74	195	598	1012	1396	19	53	92	127
K-means	437	1383	2283	3118	22	32	57	84	16	36	65	78	223	648	1061	1448	27	57	113	131
Herding	428	1411	2343	3211	21	39	59	86	21	39	67	81	202	626	1181	1489	27	54	110	134
Kcenter	433	1425	2369	3247	24	40	62	87	20	41	66	82	211	632	1211	1497	28	58	111	136
Craig	563	1617	2677	3612	46	57	69	89	40	51	76	92	334	769	1437	1618	56	78	114	133
GradMatch	571	1689	2703	3671	48	58	73	94	42	53	78	95	357	788	1462	1633	53	80	105	128
Least-CF	513	1503	2481	3329	41	48	68	92	34	49	73	89	234	689	1326	1541			-	
DFAL	961	1912	2817	3621	57	74	105	128	64	82	113	124	464	829	1523	1844			-	
Mean-STD	1523	2411	3376	4131	73	97	121	150	77	94	127	154	558	937	1587	1899	65	77	121	144
Glister	813	1846	2960	3852	78	95	117	137	69	87	103	125	464	829	1523	1844	131	280	368	584
DVRL	876	1874	3078	4127	163	251	341	432	143	237	316	408	776	1162	1469	1712	294	438	561	720
TracIn	882	1903	3142	4237	135	197	255	328	354	413	487	551	1471	1591	1737	1916	738	821	1012	1265
CG-influence	2044	3182	3569	4513	188	269	318	387	1224	1256	1297	1333	1842	2149	2452	2858	1928	1973	1990	2011
Arnoldi	2148	3273	3653	4621	187	254	323	391	1238	1268	1297	1357	1814	2162	2340	2725	1973	1990	2028	2125
TMC-shapley	3922	4832	5212	6083	241	311	387	442	1491	1519	1555	1588	2369	2731	2952	3193	2355	2417	2497	2541
CS-shapley	3984	4906	5372	6134	245	331	402	467	1497	1537	1584	1644	3188	3691	3793	3994	2472	2523	2572	2649
Full	4541				124				112				2055				230			

Table 4: Latency of different data selection methods in deep learning. Reported time (seconds) of entire model training.

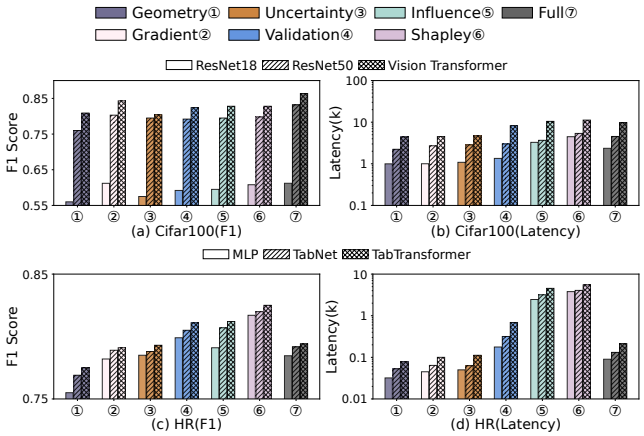


Figure 2: Model Architecture Comparison.

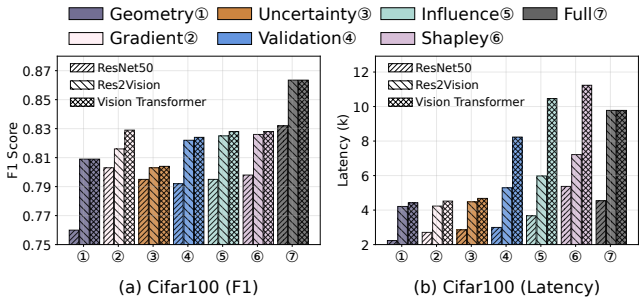


Figure 3: Model Architecture Transfer.

approximation strategies attain the highest F1 score across all categories, achieving a performance comparable to training with the full dataset [while reducing training time by half.]

For the out-of-distribution scenario, strategies like Shapley Value also improve the model accuracy. For example, Shapley Value strategies achieve the highest F1 score across all methods and even outperform full-dataset training, but require more time as model complexity increases.

Because training with complex models is time-consuming, we test whether it is effective to first select data with relatively lightweight models and then apply the selected data to train complex models. As shown in Figure 3, `Res2Vision` indicates that it uses `ResNet50` to select data but train a `Vision Transformer` model. `ResNet50` and `Vision Transformer` represent the cases that data selection and training are both performed on the same model. We observe that `Res2Vision` produces a model with the performance close to that of using `Vision Transformer` for selection, while substantially reducing computational cost. However, the gradient approximation strategies do not perform well on the largest model. This is because these strategies select weighted subsets based on model-specific parameters; consequently, the parameter distributions differ across architectures, and a good approximation on a lightweight model may not transfer effectively to a more complex one.

Key Observation 8: The effectiveness of data selection strategies remains stable across different model architectures (including SFT for LLMs), enabling the use of lightweight models for efficient subset selection without sacrificing the performance of training larger models subsequently. This improves both efficiency and effectiveness.

5 Evaluation on SFT

In this section, we evaluate both the effectiveness and efficiency for SFT. We apply 10 representative selection methods to select subsets for SFT, and evaluate the fine-tuned models on 4 downstream tasks. We thoroughly analyze the capabilities of three types of strategies: model-agnostic & task-aware strategies, model-aware & task-aware strategies and model-aware & task-agnostic strategies, focusing on three key aspects: domain-specific tasks enhancement, general task enhancement, and efficiency.

5.1 Experimental Setup

Strategies. We categorized the 10 data selection methods into three categories as discussed above: (1) Model-agnostic & task-aware

strategies (BM25 [69], DSIR [98], RDS [82]): These methods select subsets close to the validation set distribution based on different feature spaces. (2)Model-aware & task-aware strategies (LESS [96], SHED [31]): LESS uses influence functions and SHED utilizes Shapley values. (3)Model-aware & task-agnostic strategies (PPL [97], NLL [27], SelectIT [57], Superfilter [53], TAGCOS [103]): PPL, NLL, SelectIT, and Superfilter are based on uncertainty methods. TAGCOS selects a subset in the gradient space to approximate the full set.

Training datasets. In line with the experimental setting of LESS [96], we combine four widely adopted instruction tuning datasets: FLAN V2, CoT, DOLLY, and Open Assistant, containing approximately 270k samples in total. We implement an n gram filtering [26] to double-check that our training data are not contaminated by information from the downstream tasks. These datasets cover different types of tasks and prompting formats; the details are described as follows: (1) FLAN V2 [59] incorporates various prompting strategies and spans hundreds of NLP tasks that employ task balancing and augmentation techniques. (2) CoT [92] introduces intermediate reasoning steps into prompts, thereby improving performance on complex reasoning tasks. (3) DOLLY [30] includes 15k high-quality human-written instruction–response pairs, covering a wide range of tasks. (4) Open Assistant [49] comprises assistant-style dialogues organized into complete conversation trees, all of which have undergone human quality review. Overall, these datasets exhibit rich diversity in source, structure, and task distribution, providing a comprehensive corpus for data selection.

Evaluation datasets. We evaluate all data selection methods on ARC_Challenge [14], CommonsenseQA [80], GSM8K [15] and OpenBookQA [63]. Specifically, ARC_Challenge focuses on complex scientific reasoning (e.g., "Which property best explains why ice floats on water?"), CommonsenseQA on commonsense reasoning (e.g., "Where would you find a pillow?"), and GSM8K on mathematical problem-solving (e.g., "If there are 3 apples and you buy 2 more, how many do you have?"). OpenBookQA simulates open book exams to evaluate subject understanding and reasoning (e.g., "Which material is best for conducting electricity?").

Metric. Following prior works [96, 103], we report accuracy (acc) across all four tasks—ARC_Challenge, OpenBookQA, CommonsenseQA, and GSM8K.

Settings. We test data selection methods with LLAMA2-7B [36] and Mistral-7B [40], which are widely used in existing data selection strategies [96, 103]. For both models, the batch size is set to 4 and the maximum learning rate is 2e-5 with a linear decay schedule. [For SFT, we adapt LoRA for training, following the setup of LESS [96].] We train the selected data for 4 epochs on A800 GPUs. For data selection methods that require a warm-up phase (such as LESS [96] and TAGCOS [103]), we follow their settings by randomly selecting 5% of the full dataset for warm-up. After that, different selection methods are applied to choose subsets of 1%, 10%, 30%, 50% data instances from the full dataset for training. All training starts from the base model, and training configurations (e.g., batch size, LoRA configuration, number of trials) are provided in Appendix B.1.3. All evaluations are conducted under 0-shot, 2-shot and 5-shot settings, where the "shot" refers to the number of labeled examples provided in the prompt during inference.

5.2 Experimental Results and Analysis

We report 0-shot results for ARC_Challenge, CommonsenseQA, and OpenBookQA, and 2-shot results for GSM8K as our primary results. Table 5 presents the evaluation scores of each data-selection strategy across the four benchmark tasks, along with the total time of data-selection and model training. Other results w.r.t. different shots and the 50% fraction can be found in Appendix C.2.

5.2.1 Evaluation of Domain-specific Capacity. In Table 5, [the notation [·] denotes the test set and (·) denotes the validation set.] For example, the column [ARC](ARC) represents the performance of different strategies when both the validation and test set are ARC_Challenge. Remind that the validation set and test set do not overlap. Beside, note that (·) is only for task-aware strategies (including DSIR, RDS, BM25, LESS and SHED). For task-agnostic strategies (including Superfilter, TAGCOS, PPL, NLL and SelectIT), we can ignore (·) and only consider [·] because they do not involve a validation set.

We find that model-aware & task-aware strategies (e.g., LESS, marked in blue in Table 5) achieve the best overall performance on domain-specific tasks. For example, LESS outperforms PPL by more than 7% at the 30% fraction on CommonsenseQA. This is because model-aware & task-agnostic strategies do not leverage downstream task information, making them less effective when task-specific knowledge is required. Similarly, LESS achieves 6% higher performance than RDS at the 30% fraction on ARC_Challenge, as model-agnostic & task-aware strategies rely solely on the feature space of the data without incorporating model feedback, limiting their ability to identify the most informative instances for the model.

Key Observation 9: For domain-specific tasks, the task-aware strategies are more effective than task-agnostic strategies by referring to validation dataset during data selection, leading to more effective data selection for that domain. Among task-aware strategies, model-aware strategies achieve the best performance because they focus on identifying and selecting the data instances within the target domain that have the most significant impact on the model's performance.

In particular, although SHED is a task-aware & model-aware strategy, it does not perform well. The primary reason is that SHED approximates each cluster's Shapley value solely from its centroid, which fails to capture the contributions of each individual data instance. Combined with the expensive Shapley Value computation, SHED remains the most time-consuming method despite its clustering heuristic.

5.2.2 Overall general Effectiveness Comparison In Table 6, the notation [CQA](ARC) indicates that data selection is performed with ARC_challenge as the validation dataset and evaluated on CommonsenseQA. Unlike the previous subsection, for all task-aware strategies that require the validation dataset, we use ARC_challenge as the validation set and evaluate it on different tasks to evaluate the generalization ability of different strategies. Additionally, we report the results with other tasks as the validation dataset in Appendix C.2.

The experimental results indicate that model-aware & task-agnostic strategies (e.g., Superfilter) achieve better performance than

Method	[ARC](ARC)			[CQA](CQA)			[OQA](OQA)			[GSM8k](GSM8k)			Selection + Training Time (GPU*h)			
	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.5
Random	0.47	0.48	0.50	0.41	0.47	0.53	0.418	0.444	0.440	0.12	0.16	0.20	0+0.5	0+5	0+16	0+26.5
DSIR	0.49	0.50	0.50	0.44	0.40	0.51	0.450	0.448	0.454	0.13	0.17	0.21	0+0.5	0+5	0+16	0+26.5
RDS	**0.51**	0.51	0.53	0.44	0.52	0.58	0.442	0.448	0.450	0.10	0.11	0.15	3.5+0.5	3.5+5	3.5+16	3.5+26.5
BM25	0.50	0.49	0.50	0.44	0.56	0.60	0.458	0.454	0.444	0.13	0.20	0.24	0+0.5	0+5	0+16	0+26.5
LESS	**0.51**	**0.54**	**0.56**	**0.52**	**0.57**	**0.64**	**0.464**	**0.472**	**0.481**	**0.16**	**0.23**	**0.27**	56+0.5	56+5	56+16	56+26.5
SHED	0.50	0.50	0.49	0.41	0.49	0.53	0.437	0.444	0.452	0.13	0.16	0.20	183+0.5	183+5	183+16	183+26.5
Superfilter	**0.51**	0.52	0.52	0.50	0.53	0.58	0.461	0.454	0.462	0.15	0.21	0.24	36+0.5	36+5	36+16	36+26.5
TAGCOS	0.48	0.49	0.49	0.47	0.50	0.49	0.454	0.458	0.459	0.14	0.19	0.23	56+0.5	56+5	56+16	56+26.5
PPL	**0.51**	0.51	0.51	0.51	0.50	0.56	0.454	0.464	0.471	0.15	0.20	0.22	20+0.5	20+5	20+16	20+26.5
NLL	0.50	0.51	0.51	0.50	0.52	0.57	0.454	0.458	0.460	0.14	0.18	0.22	20+0.5	20+5	20+16	20+26.5
SelectIT	0.49	0.48	0.49	0.46	0.47	0.57	0.462	0.463	0.462	0.15	0.21	0.23	4+0.5	4+5	4+16	4+26.5
Full		0.52			0.59			0.461			0.27				0+53	

Table 5: Performance based on domain evaluation and data selection + training time (GPU*h) of different data selection methods. Model-agnostic & task-aware strategies are marked in orange, model-aware & task-aware strategies in blue, and model-aware & task-agnostic strategies in green. We run each experiment three times and report the average.

Method	[ARC](ARC)			[CQA](ARC)			[OQA](ARC)			[GSM8k](ARC)			Average		
	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3
Random	0.47	0.48	0.50	0.41	0.47	0.53	0.418	0.444	0.440	0.12	0.16	0.20	0.35	0.39	0.42
DSIR	0.49	0.50	0.50	0.40	0.42	0.50	0.447	0.444	0.450	0.14	0.17	0.20	0.37	0.38	0.41
RDS	**0.51**	0.51	0.53	0.38	0.47	0.56	0.441	0.443	0.436	0.10	0.10	0.13	0.36	0.38	0.41
BM25	0.50	0.49	0.50	0.42	0.47	0.53	0.454	0.447	0.450	0.13	0.15	0.21	0.38	0.39	0.42
LESS	**0.51**	**0.54**	**0.56**	0.44	0.46	0.53	0.456	0.460	0.444	0.13	0.17	0.23	0.38	0.41	0.44
SHED	0.50	0.50	0.49	0.40	0.47	0.51	0.430	0.445	0.453	0.13	0.17	0.22	0.37	0.40	0.42
Superfilter	**0.51**	0.52	0.55	0.50	**0.53**	**0.58**	0.461	0.454	0.462	**0.15**	**0.21**	**0.24**	**0.41**	**0.43**	**0.46**
TAGCOS	0.48	0.49	0.49	0.47	0.50	0.49	0.454	0.458	0.459	0.14	0.19	0.23	0.39	0.41	0.42
PPL	**0.51**	0.51	0.51	**0.51**	0.50	0.56	0.454	**0.464**	**0.471**	**0.15**	0.20	0.22	**0.41**	0.42	0.44
NLL	0.50	0.51	0.51	0.50	0.52	0.57	0.454	0.458	0.460	0.14	0.18	0.22	0.40	0.42	0.44
SelectIT	0.49	0.48	0.49	0.46	0.47	0.57	**0.462**	0.463	0.462	**0.15**	**0.21**	0.23	0.39	0.41	0.44
Full		0.52			0.59			0.46			0.27			0.46	

Table 6: Performance based on general evaluation of different data selection methods. The validation set is ARC_challenge. We run each experiment three times and report the average.

model-aware & task-aware strategies. For example, Superfilter achieves an average score 5% higher than LESS at the 30% fraction, as task-aware strategies mostly focus on validation dataset. This limits the diversity of their data selection, resulting in relatively modest performance in general evaluation. Among task-agnostic strategies, Superfilter, which selects data based on IFD scores, strengthens the model's instruction following capability, thereby achieving superior generalization performance compared to other task-agnostic strategies. NLL and PPL have nearly identical performance due to their similar computation. TAGCOS leverages the coreset selection approach to approximate the overall data distribution at the gradient level, achieving comparable performance to full-dataset training while using only a weighted subset. However, since the full dataset contains low-quality data, the performance of TAGCOS still falls short compared to methods based on uncertainty estimation.

Key Observation 10: Model-aware & task-agnostic strategies exhibit better generalization because they focus on selecting data with higher diversity, which enhances the model's overall capability instead of restricting improvements to a specific domain.

5.2.3 Overall Efficiency Comparison In terms of computational efficiency, model-agnostic & task-aware methods are the most efficient, as they avoid forward and backward propagation in LLMs with millions of parameters. For other methods, uncertainty-based methods like PPL, NLL, and SelectIT require only forward propagation, leading to slightly longer selection time than the former one. Superfilter utilizes IFD for data selection. Since calculating IFD requires two passes of PPL, Superfilter nearly doubles the time compared to PPL. LESS and TAGCSOS perform data selection in the gradient space. The primary time complexity for these methods lies in computing data gradients for each data instances, making their data selection the most time-consuming. SHED employs Shapley values for data selection. Although representative data instances are selected via clustering to avoid the computation of Shapley values for each instance, calculating Shapley values still requires extensive model retraining. Consequently, SHED has the longest data selection time among all evaluated methods.

6 Conclusion & Future Directions

This work presents a comprehensive review and evaluation of data selection strategies for supervised deep learning and SFT for LLMs. By systematically reviewing and categorizing existing methods,

we highlight the diversity of approaches that target training efficiency or effectiveness. Then, through a unified testbed and extensive experiments across diverse datasets and models, we show the advantages/disadvantages of existing data selections and provide guidelines for choosing the most appropriate strategy for different scenarios.

Promising future directions might include (1) Given fine-grained user requirements, e.g., time constraint, accuracy requirement, computation resources constraint, etc., how to select the most appropriate data selection strategy with the optimization goal of effectiveness or efficiency. (2) In real-life scenarios, training data is always not sufficient, regardless of deep learning and SFT. To address this issue, data synthesis is potentially an effective strategy, but the quality of the synthesized data cannot be guaranteed. Therefore, it is important to study how to select data instances from synthesized data for downstream tasks. (3) Investigation of how to combine the techniques of efficiency and effectiveness strategies to judiciously develop a strategy that can achieve high efficacy given a large amount of candidate data.

References

[1] Peyman Afshani and Chris Schwiegelshohn. [n.d.]. Optimal Coresets for Low-Dimensional Geometric Median. In Forty-first International Conference on Machine Learning.

[2] Sharat Agarwal, Himanshu Arora, Sandeep Anand, and Chetan Arora. 2020. Contextual Diversity for Active Learning. Cornell University - arXiv,Cornell University - arXiv (Aug 2020).

[3] Md Atik Ahamed and Qiang Cheng. 2024. MambaTab: A Plug-and-Play Model for Learning Tabular Data. arXiv:2401.08867 [cs.LG] https://arxiv.org/abs/2401.08867

[4] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-Generation Hyperparameter Optimization Framework. In The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2623–2631.

[5] Alon Albalak, Yanai Elazar, Sang Michael Xie, Shayne Longpre, Nathan Lambert, Xinyi Wang, Niklas Muennighoff, Bairu Hou, Liangming Pan, Haewon Jeong, et al. 2024. A survey on data selection for language models. arXiv preprint arXiv:2402.16827 (2024).

[6] Sercan O. Arik and Tomas Pfister. 2020. TabNet: Attentive Interpretable Tabular Learning. arXiv:1908.07442 [cs.LG] https://arxiv.org/abs/1908.07442

[7] Kyriakos Axiotis, Vincent Cohen-Addad, Monika Henzinger, Sammy Jerome, Vahab Mirrokni, David Saulpic, David Woodruff, and Michael Wunder. 2024. Data-Efficient Learning via Clustering-Based Sensitivity Sampling: Foundation Models and Beyond. arXiv:2402.17327 [cs.LG] https://arxiv.org/abs/2402.17327

[8] Nikhil Bansal, Vincent Cohen-Addad, Milind Prabhu, David Saulpic, and Chris Schwiegelshohn. 2024. Sensitivity Sampling for k-Means: Worst Case and Stability Optimal Coreset Bounds. arXiv:2405.01339 [cs.DS] https://arxiv.org/abs/2405.01339

[9] Ella Bingham and Heikki Mannila. 2001. Random projection in dimensionality reduction: applications to image and text data (KDD '01). Association for Computing Machinery, New York, NY, USA, 245–250. https://doi.org/10.1145/502512.502546

[10] Jock Blackard. 1998. Covertype. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C50K5N.

[11] Chengliang Chai, Jiabin Liu, Nan Tang, Guoliang Li, and Yuyu Luo. 2022. Selective data acquisition in the wild for model charging. Proceedings of the VLDB Endowment (Mar 2022), 1466–1478. https://doi.org/10.14778/3523210.3523223

[12] Chengliang Chai, Jiayi Wang, Nan Tang, Ye Yuan, Jiabin Liu, Yuhao Deng, and Guoren Wang. 2023. Efficient Coreset Selection with Cluster-based Methods. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Long Beach, CA, USA) (KDD '23). Association for Computing Machinery, New York, NY, USA, 167–178. https://doi.org/10.1145/3580305.3599326

[13] Yutian Chen, Max Welling, and Alex Smola. 2012. Super-samples from kernel herding. arXiv preprint arXiv:1203.3472 (2012).

[14] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457 [cs.AI] https://arxiv.org/abs/1803.05457

[15] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] https://arxiv.org/abs/2110.14168

[16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929 [cs.CV] https://arxiv.org/abs/2010.11929

[17] Melanie Ducoffe and Frederic Precioso. 2018. Adversarial active learning for deep networks: a margin based approach. arXiv preprint arXiv:1802.09841 (2018).

[18] Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. arXiv:1506.02142 [stat.ML] https://arxiv.org/abs/1506.02142

[19] Yarin Gal, Riashat Islam, and Zoubin Ghahramani. 2017. Deep bayesian active learning with image data. In International conference on machine learning. PMLR, 1183–1192.

[20] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2017. Domain-Adversarial Training of Neural Networks. 189–209. https://doi.org/10.1007/978-3-319-58347-1_10

[21] Jakob Gawlikowski, Cedrique Rovile Njieutcheu Tassi, Mohsin Ali, Jongseok Lee, Matthias Humt, Jianxiang Feng, Anna Kruspe, Rudolph Triebel, Peter Jung, Ribana Roscher, Muhammad Shahzad, Wen Yang, Richard Bamler, and Xiao Xiang Zhu. 2022. A Survey of Uncertainty in Deep Neural Networks. arXiv:2107.03342 [cs.LG] https://arxiv.org/abs/2107.03342

[22] Jakob Gawlikowski, Cedrique Rovile Njieutcheu Tassi, Mohsin Ali, Jongseok Lee, Matthias Humt, Jianxiang Feng, Anna Kruspe, Rudolph Triebel, Peter Jung, Ribana Roscher, Muhammad Shahzad, Wen Yang, Richard Bamler, and Xiao Xiang Zhu. 2022. A Survey of Uncertainty in Deep Neural Networks. arXiv:2107.03342 [cs.LG] https://arxiv.org/abs/2107.03342

[23] Amirata Ghorbani and James Zou. 2019. Data shapley: Equitable valuation of data for machine learning. In International conference on machine learning. PMLR, 2242–2251.

[24] Curtis Godwin. 2022. Incorporating Task-Agnostic Information in Task-Based Active Learning Using a Variational Autoencoder.

[25] Chengcheng Guo, Bo Zhao, and Yanbing Bai. 2022. Deepcore: A comprehensive library for coreset selection in deep learning. In International Conference on Database and Expert Systems Applications. Springer, 181–195.

[26] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 [cs.SE] https://arxiv.org/abs/2401.14196

[27] Jindong Han, Hao Liu, Jun Fang, Naiqiang Tan, and Hui Xiong. 2025. Automatic Instruction Data Selection for Large Language Models via Uncertainty-Aware Influence Maximization. In Proceedings of the ACM on Web Conference 2025 (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 4969–4979. https://doi.org/10.1145/3696410.3714817

[28] Sariel Har-Peled and Akash Kushal. 2005. Smaller coresets for k-median and k-means clustering. In Proceedings of the twenty-first annual symposium on Computational geometry. 126–134.

[29] Yuto Harada, Yusuke Yamauchi, Yusuke Oda, Yohei Oseki, Yusuke Miyao, and Yu Takagi. 2025. Massive Supervised Fine-tuning Experiments Reveal How Data, Layer, and Training Factors Shape LLM Alignment Quality. arXiv:2506.14681 [cs.CL] https://arxiv.org/abs/2506.14681

[30] Shirley Anugrah Hayati, Taehee Jung, Tristan Bodding-Long, Sudipta Kar, Abhinav Sethy, Joo-Kyung Kim, and Dongyeop Kang. 2025. Chain-of-Instructions: Compositional Instruction Tuning on Large Language Models. arXiv:2402.11532 [cs.CL] https://arxiv.org/abs/2402.11532

[31] Yexiao He, Ziyao Wang, Zheyu Shen, Guoheng Sun, Yucong Dai, Yongkai Wu, Hongyi Wang, and Ang Li. 2024. SHED: Shapley-Based Automated Dataset Refinement for Instruction Fine-Tuning. arXiv preprint arXiv:2405.00705 (2024).

[32] Alex Holub, Pietro Perona, and Michael C. Burl. 2008. Entropy-based active learning for object recognition. In 2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops. https://doi.org/10.1109/cvprw.2008.4563068

[33] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL] https://arxiv.org/abs/2106.09685

[34] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL] https://arxiv.org/abs/2106.09685

[35] Xin Huang, Ashish Khetan, Milan Cvitkovic, and Zohar Karnin. 2020. TabTransformer: Tabular Data Modeling Using Contextual Embeddings. arXiv:2012.06678 [cs.LG] https://arxiv.org/abs/2012.06678

[36] Kevin Stone Angela Fan Hugo Touvron, Louis Martin. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] https://arxiv.org/abs/2307.09288

[37] Hamish Ivison, Muru Zhang, Faeze Brahman, Pang Wei Koh, and Pradeep Dasigi. 2025. Large-Scale Data Selection for Instruction Tuning. arXiv:2503.01807 [cs.CL] https://arxiv.org/abs/2503.01807

[38] Ruoxi Jia, David Dao, Boxin Wang, FrancesAnn Hubis, Nick Hynes, Nezihe-Merve Gürel, Bo Li, Ce Zhang, Dawn Song, and CostasJ. Spanos. 2019. Towards Efficient Data Valuation Based on the Shapley Value. International Conference on Artificial Intelligence and Statistics,International Conference on Artificial Intelligence and Statistics (Apr 2019).

[39] Ruoxi Jia, David Dao, Boxin Wang, Frances Ann Hubis, Nezihe Merve Gurel, Bo Li, Ce Zhang, Costas Spanos, and Dawn Song. 2019. Efficient task-specific data valuation for nearest neighbor algorithms. Proceedings of the VLDB Endowment (Jul 2019), 1610–1623. https://doi.org/10.14778/3342263.3342637

[40] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[41] Ayrton San Joaquin, Bin Wang, Zhengyuan Liu, Nicholas Asher, Brian Lim, Philippe Muller, and Nancy F. Chen. 2024. In2Core: Leveraging Influence Functions for Coreset Selection in Instruction Finetuning of Large Language Models. arXiv:2408.03560 [cs.LG] https://arxiv.org/abs/2408.03560

[42] Ajay J Joshi, Fatih Porikli, and Nikolaos Papanikolopoulos. 2009. Multi-class active learning for image classification. In 2009 ieee conference on computer vision and pattern recognition. IEEE, 2372–2379.

[43] Michael Kampffmeyer, Arnt-Borre Salberg, and Robert Jenssen. 2016. Semantic Segmentation of Small Objects and Modeling of Uncertainty in Urban Remote Sensing Images Using Deep Convolutional Neural Networks. In 2016 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). https://doi.org/10.1109/cvprw.2016.90

[44] Krishnateja Killamsetty, Dheeraj Bhat, Ganesh Ramakrishnan, and Rishabh Iyer. 2022. CORDS: COResets and Data Subset Selection for Efficient Learning. https://github.com/decile-team/cords Released 2022-03-23; accessed 2025-05-13.

[45] Krishnateja Killamsetty, Sivasubramanian Durga, Ganesh Ramakrishnan, Abir De, and Rishabh Iyer. 2021. Grad-match: Gradient matching based data subset selection for efficient deep model training. In International Conference on Machine Learning. PMLR, 5464–5474.

[46] Krishnateja Killamsetty, Durga Sivasubramanian, Ganesh Ramakrishnan, and Rishabh Iyer. 2021. Glister: Generalization based data subset selection for efficient and robust learning. In Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35. 8110–8118.

[47] Pang Wei Koh and Percy Liang. 2017. Understanding black-box predictions via influence functions. In International conference on machine learning. PMLR, 1885–1894.

[48] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).

[49] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi-Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Richárd Nagyfi, Shahul ES, Sameer Suri, David Glushkov, Arnav Dantuluri, Andrew Maguire, Christoph Schuhmann, Huu Nguyen, and Alexander Mattick. 2023. OpenAssistant Conversations – Democratizing Large Language Model Alignment. arXiv:2304.07327 [cs.CL] https://arxiv.org/abs/2304.07327

[50] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating Training Data Makes Language Models Better. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 8424–8445.

[51] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? Proceedings of the VLDB Endowment (Nov 2015), 204–215. https://doi.org/10.14778/2850583.2850594

[52] David D. Lewis and William A. Gale. 1994. A Sequential Algorithm for Training Text Classifiers. In Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval. 3–12.

[53] Ming Li, Yong Zhang, Shwai He, Zhitao Li, Hongyu Zhao, Jianzong Wang, Ning Cheng, and Tianyi Zhou. 2024. Superfiltering: Weak-to-Strong Data Filtering for Fast Instruction-Tuning. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 14255–14273. https://aclanthology.org/2024.acl-long.769

[54] Ming Li, Yong Zhang, Zhitao Li, Jiuhai Chen, Lichang Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and Jing Xiao. 2024. From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning. arXiv:2308.12032 [cs.CL] https://arxiv.org/abs/2308.12032

[55] Jie Liu, Peizheng Wang, and Chao Wu. 2023. Data valuation: The partial ordinal Shapley value for machine learning. arXiv:2305.01660 [cs.LG] https://arxiv.org/abs/2305.01660

[56] Jiabin Liu, Fu Zhu, Chengliang Chai, Yuyu Luo, and Nan Tang. 2021. Automatic data acquisition for deep learning. Proc. VLDB Endow. 14, 12 (July 2021), 2739–2742. https://doi.org/10.14778/3476311.3476333

[57] Liangxin Liu, Xuebo Liu, Derek F. Wong, Dongfang Li, Ziyi Wang, Baotian Hu, and Min Zhang. 2025. SelectIT: Selective Instruction Tuning for LLMs via Uncertainty-Aware Self-Reflection. arXiv:2402.16705 [cs.CL] https://arxiv.org/abs/2402.16705

[58] Ziche Liu, Rui Ke, Yajiao Liu, Feng Jiang, and Haizhou Li. 2025. Take the essence and discard the dross: A Rethinking on Data Selection for Fine-Tuning Large Language Models. arXiv:2406.14115 [cs.CL] https://arxiv.org/abs/2406.14115

[59] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. 2023. The Flan Collection: Designing Data and Methods for Effective Instruction Tuning. arXiv:2301.13688 [cs.AI] https://arxiv.org/abs/2301.13688

[60] Keming Lu, Hongyi Yuan, Zheng Yuan, Runji Lin, Junyang Lin, Chuanqi Tan, Chang Zhou, and Jingren Zhou. 2023. InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. arXiv:2308.07074 [cs.CL] https://arxiv.org/abs/2308.07074

[61] Xuan Luo, Jian Pei, Zicun Cong, and Cheng Xu. 2022. On shapley value in data assemblage under independent utility. Proc. VLDB Endow. 15, 11 (July 2022), 2761–2773. https://doi.org/10.14778/3551793.3551829

[62] Hlib Mavrodiev. 2017. London Bike Sharing Dataset. https://www.kaggle.com/datasets/hmavrodiev/london-bike-sharing-dataset.

[63] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. arXiv:1809.02789 [cs.CL] https://arxiv.org/abs/1809.02789

[64] Baharan Mirzasoleiman, Jeff Bilmes, and Jure Leskovec. 2020. Coresets for data-efficient training of machine learning models. In International Conference on Machine Learning. PMLR, 6950–6960.

[65] Rohit Misra. 2022. News Category Dataset. https://www.kaggle.com/datasets/rmisra/news-category-dataset.

[66] Raymond H. Myers. 1974. Elementary Applied Statistics. Technometrics (Feb 1974), 137–137. https://doi.org/10.1080/00401706.1974.10489161

[67] Y. C. Pati, R. Rezaiifar, and P. S. Krishnaprasad. 1993. Orthogonal Matching Pursuit: Recursive Function Approximation with Applications to Wavelet Decomposition. In Proceedings of the 27th Asilomar Conference on Signals, Systems and Computers. 40–44.

[68] Garima Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. 2020. Estimating training data influence by tracing gradient descent. Advances in Neural Information Processing Systems 33 (2020), 19920–19930.

[69] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. Found. Trends Inf. Retr. 3, 4 (April 2009), 333–389. https://doi.org/10.1561/1500000019

[70] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. ImageNet Large Scale Visual Recognition Challenge. arXiv:1409.0575 [cs.CV] https://arxiv.org/abs/1409.0575

[71] Andrea Schioppa, Polina Zablotskaia, David Vilar, and Artem Sokolov. 2022. Scaling up influence functions. In Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36. 8179–8186.

[72] Stephanie Schoch, Ritwick Mishra, and Yangfeng Ji. 2023. Data Selection for Fine-tuning Large Language Models Using Transferred Shapley Values. arXiv:2306.10165 [cs.CL] https://arxiv.org/abs/2306.10165

[73] Stephanie Schoch, Haifeng Xu, and Yangfeng Ji. 2022. CS-Shapley: class-wise Shapley values for data valuation in classification. Advances in Neural Information Processing Systems 35 (2022), 34574–34585.

[74] Ozan Sener and Silvio Savarese. 2017. Active learning for convolutional neural networks: A core-set approach. arXiv preprint arXiv:1708.00489 (2017).

[75] Sofia Serrano, Zander Brumbaugh, and Noah A. Smith. 2023. Language Models: A Guide for the Perplexed. arXiv:2311.17301 [cs.CL] https://arxiv.org/abs/2311.17301

[76] Yunfan Shao, Linyang Li, Zhaoye Fei, Hang Yan, Dahua Lin, and Xipeng Qiu. 2024. Balanced Data Sampling for Language Model Training with Clustering. arXiv:2402.14526 [cs.CL] https://arxiv.org/abs/2402.14526

[77] Lloyd S. Shapley. 1988. A value for n-person games. 31–40. https://doi.org/10.1017/cbo9780511528446.003

[78] Samarth Sinha, Han Zhang, Anirudh Goyal, Yoshua Bengio, Hugo Larochelle, and Augustus Odena. 2019. Small-GAN: Speeding Up GAN Training Using Core-sets. arXiv: Machine Learning,arXiv: Machine Learning (Oct 2019).

[79] Haifeng Sun, Yu Xiong, Runze Wu, Xinyu Cai, Changjie Fan, Lan Zhang, and Xiang-Yang Li. 2025. Fast-DataShapley: Neural Modeling for Training Data Valuation. arXiv:2506.05281 [cs.LG] https://arxiv.org/abs/2506.05281

[80] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. arXiv:1811.00937 [cs.CL] https://arxiv.org/abs/1811.00937

[81] TransferLab team. 2025. pyDVL (v0.10.0). https://doi.org/10.5281/zenodo.15188659

[82] Chun-Hua Tsai and Peter Brusilovsky. 2019. Evaluating Visual Explanations for Similarity-Based Recommendations: User Perception and Performance. In Proceedings of the 27th ACM Conference on User Modeling, Adaptation and Personalization (Larnaca, Cyprus) (UMAP '19). Association for Computing Machinery, New York, NY, USA, 22–30. https://doi.org/10.1145/3320435.3320465

[83] Joannès Vermorel and Mehryar Mohri. 2005. Multi-armed Bandit Algorithms and Empirical Evaluation. 437–448. https://doi.org/10.1007/11564096_42

[84] Thien Nhan Vo. 2025. Active Learning for Multi-class Image Classification. arXiv:2505.06825 [cs.CV] https://arxiv.org/abs/2505.06825

[85] Jiayi Wang, Chengliang Chai, Nan Tang, Jiabin Liu, and Guoliang Li. 2022. Coresets over multiple tables for feature-rich and data-efficient machine learning. Proc. VLDB Endow. 16, 1 (Sept. 2022), 64–76. https://doi.org/10.14778/3561261.3561267

[86] Jiayi Wang, Chengliang Chai, Nan Tang, Jiabin Liu, and Guoliang Li. 2022. Coresets over multiple tables for feature-rich and data-efficient machine learning. Proceedings of the VLDB Endowment 16, 1 (2022), 64–76.

[87] Jingtan Wang, Xiaoqiang Lin, Rui Qiao, Chuan-Sheng Foo, and Bryan Kian Hsiang Low. 2024. Helpful or Harmful Data? Fine-tuning-free Shapley Attribution for Explaining Language Model Predictions. In Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research), Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.), Vol. 235. PMLR, 50960–50991. https://proceedings.mlr.press/v235/wang24aq.html

[88] JiachenT Wang, Tong Wu, UC Berkeley, Prateek Mittal, and Ruoxi Jia. [n.d.]. GREATS: Online Selection of High-Quality Data for LLM Training in Every Iteration. ([n. d.]).

[89] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2024. Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv:2212.03533 [cs.CL] https://arxiv.org/abs/2212.03533

[90] Sheng Wang, Yuan Sun, and Zhifeng Bao. 2020. On the efficiency of k-means clustering: Evaluation, optimization, and algorithm selection. arXiv preprint arXiv:2010.06654 (2020).

[91] Tingting Wang, Shixun Huang, Zhifeng Bao, J Shane Culpepper, Volkan Dedeoglu, and Reza Arablouei. 2024. Optimizing Data Acquisition to Enhance Machine Learning Performance. Proceedings of the VLDB Endowment 17, 6 (2024), 1310–1323.

[92] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL] https://arxiv.org/abs/2201.11903

[93] Max Welling. 2009. Herding dynamical weights to learn. In Proceedings of the 26th annual international conference on machine learning. 1121–1128.

[94] Yang Wu, Huayi Zhang, Yizheng Jiao, Lin Ma, Xiaozhong Liu, Jinhong Yu, Dongyu Zhang, Dezhi Yu, and Wei Xu. 2024. ROSE: A Reward-Oriented Data Selection Framework for LLM Task-Specific Instruction Tuning. arXiv:2412.00631 [cs.LG] https://arxiv.org/abs/2412.00631

[95] Haocheng Xia, Jinfei Liu, Jian Lou, Zhan Qin, Kui Ren, Yang Cao, and Li Xiong. 2023. Equitable Data Valuation Meets the Right to Be Forgotten in Model Markets. Proc. VLDB Endow. 16, 11 (July 2023), 3349–3362. https://doi.org/10.14778/3611479.3611531

[96] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In International Conference on Machine Learning (ICML).

[97] Tingyu Xia, Bowen Yu, Kai Dang, An Yang, Yuan Wu, Yuan Tian, Yi Chang, and Junyang Lin. 2024. Rethinking Data Selection at Scale: Random Selection is Almost All You Need. arXiv:2410.09335 [cs.CL] https://arxiv.org/abs/2410.09335

[98] Sang Michael Xie, Shibani Santurkar, Tengyu Ma, and Percy Liang. 2023. Data Selection for Language Models via Importance Resampling. Advances in Neural Information Processing Systems (NeurIPS) (2023).

[99] Shuo Yang, Zhe Cao, Sheng Guo, Ruiheng Zhang, Ping Luo, Shengping Zhang, and Liqiang Nie. 2024. Mind the Boundary: Coreset Selection via Reconstructing the Decision Boundary. In Forty-first International Conference on Machine Learning.

[100] Yu Yang, Hao Kang, and Baharan Mirzasoleiman. 2023. Towards Sustainable Learning: Coresets for Data-efficient Deep Learning. (Jun 2023).

[101] Jinsung Yoon, Sercan Arik, and Tomas Pfister. 2020. Data valuation using reinforcement learning. In International Conference on Machine Learning. PMLR, 10842–10851.

[102] Huayi Zhang, Binwei Yan, Lei Cao, Samuel Madden, and Elke Rundensteiner. 2024. MetaStore: Analyzing Deep Learning Meta-Data at Scale. Proc. VLDB Endow. 17, 6 (Feb. 2024), 1446–1459. https://doi.org/10.14778/3648160.3648182

[103] Jipeng Zhang, Yaxuan Qin, Renjie Pi, Weizhong Zhang, Rui Pan, and Tong Zhang. 2025. TAGCOS: Task-agnostic Gradient Clustered Coreset Selection for Instruction Tuning Data. In Findings of the Association for Computational Linguistics: NAACL 2025, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 4671–4686. https://aclanthology.org/2025.findings-naacl.264/

A Strategies Detail

A.1 Geometry-based Strategy

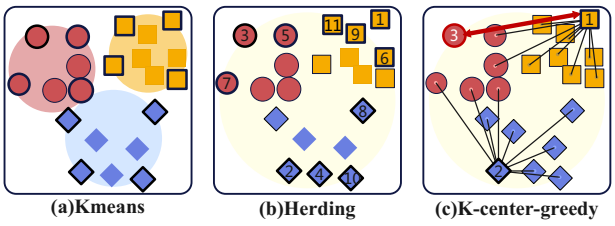


Figure 4: Geometry-based Strategy

K-means based methods [28, 90] partition the dataset D into K clusters in feature space. Then, all data instances are allocated to their nearest cluster centroid. Afterwards, suppose that we want to select $|S|$ instances. As shown in Figure 4-(a), we can select $\frac{s \times C}{N}$ instances farthest from the centroid of each cluster [90]. Alternatively, we can also sample $\frac{s \times C}{N}$ instances proportional to their distance from the centroid [8]. The overall time complexity is $O(NKd + S)$ where $K \ll N$ and d represents the dimension of features. Here, $O(NKd)$ corresponds to the typical k-means clustering and $O(S)$ is associated with the final selection of S representative instances.

Herding-based methods [13, 93] greedily select instances to minimize the squared distance between the mean embedding of the selected subset μ_S and that of the full dataset μ_D in feature space, i.e., $\arg\min_{S \subseteq D} \|\mu_D - \mu_S\|^2_{\mathcal{H}}$. This approach selects instances in an iterative manner. As illustrated in Figure 4-(b), the algorithm begins with a random data added in the selected set. At each step, it selects the instance that most reduces the difference between the mean of the selected instances (shown by the circled instances) and the mean of the full dataset (all instances). After each selection, the mean of the selected set is updated, and the process repeats until all s instances are selected. The overall time complexity is $O(Nsd)$. At each selection iteration, the method scans all N data instances. For each instance, it computes the distance with a d-dimensional space. In practice, since s is often much larger than K, herding-based methods typically incur higher computation cost than K-means ones.

K-center-greedy [74] also selects data instances iteratively. The basic idea is to always select the next instance far from the instances in the already selected set. Note that the distance between an instance and the set is defined as the minimum distance between the instance and instances in the set. To be specific, it first computes the centroid of all data instances in the feature space. Then, it selects the data instance (labeled by 1) that is the farthest from this centroid, as shown in Fig. 4. Next, since currently the set only contains one instance, the algorithm identifies the next instance (labeled as 2) that is the farthest from the instance 1. Afterwards, we compute the minimal distance of each instance that has not been selected and pick the one with the the largest minimal distance as the next instance (labeled as 3). As shown in the figure, the black lines represent the minimal distances for each unselected instance. The red arrow highlights the largest minimal distance. This process repeats until all s instances are selected. The overall time complexity is $O(Nsd)$, as each of the s selection steps scans all N data instances and updates minimal distances in d-dimensional space.

A.2 Gradient Strategy

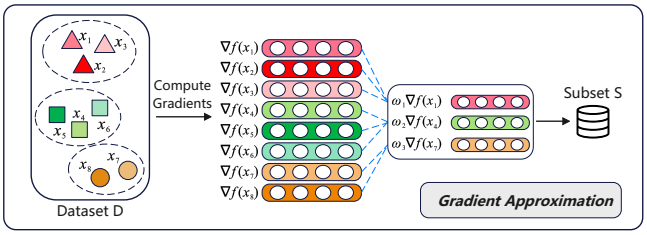


Figure 5: Gradient Approximation

`Craig` [64] is a typical algorithm of gradient approximation, which selects a subset iteratively during training. At each iteration, `Craig` trains over the selected subset, obtains the gradient information of the full dataset and updates the subset based on the gradient information. `Craig` updates the subset by solving Equation 3. First, it computes pairwise similarities of gradients between all data instances, with a time complexity of $O(N^2P)$, where P is the number of model parameters. Then, it focuses on minimizing the upper bound of the gradient approximation error, which can be formulated as an NP-hard problem with a submodular property. Hence, `Craig` iteratively selects s data instances via a greedy algorithm with an approximation error. At each selection iteration, it samples k instances from D, picks the one that can most largely reduce the gradient approximation error and adds it to the currently selected subset. Since computing error reduction requires iterating D, the overall time complexity is $O(N^2P + kNs)$.

`GradMatch` [45] is another representative algorithm that selects a subset using Equation 3. However, unlike `Craig` that minimizes the upper bound of Equation 3, `GradMatch` solves the subset selection problem more directly via orthogonal matching pursuit (OMP) [67]. The weights in this method can be real-valued and non-negative, in contrast to the integer weights in `Craig`. The overall time complexity is $O(NP + Ns^3P)$. Therefore, `GradMatch` is generally more accurate than `Craig` due to its fine-grained gradient matching in the parameter space, but its computation cost is high.

A.3 Uncertainty-based Strategy

Uncertainty-based strategies score and select data instances according to various measures of model uncertainty; we represent the uncertainty score by $U(x)$, and there exist five ways to compute it:

Least Confidence[42]. This strategy measures uncertainty as one minus the maximum predicted probability for any class: $U(x) = 1 - \max_y p(y|x)$. A lower maximum probability corresponds to greater uncertainty. The intuition is that if the model's top choice is not confident, the sample is likely to be informative for training. The time complexity is $O(NF)$, as computing the least confidence score requires one forward pass per instance, while the additional cost for selecting the top s instances is negligible.

Decision Boundary Uncertainty[17, 99] directly estimates uncertainty by measuring the distance from each instance to the decision boundary [17, 99]. As illustrated in Figure 6, this approach

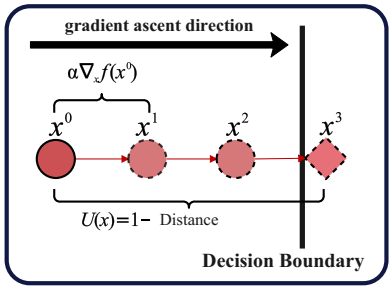


Figure 6: Decision Boundary

iteratively updates x^0 along the gradient ascent direction with $x^{i+1} = x^i + \alpha\nabla_x f(x^i)$ until the decision boundary is reached. The value $U(x) = 1 - distance$ quantifies uncertainty, with smaller distance indicating higher uncertainty. This requires M backward steps per instance, resulting in a complexity of $O(NMB)$.

Mean STD[42]. The mean standard deviation (Mean STD) method evaluates uncertainty by averaging the standard deviation of the predicted probabilities across all classes: $U(x) = \frac{1}{C}\sum_{i=1}^{C}\sigma_x^i$, where σ_x^i denotes the standard deviation of the predicted probability for class i over data instance x. Instances with higher mean standard deviation indicate higher uncertainty and are therefore prioritized for selection. The time complexity is $O(NMF)$, as each of the N instances requires M forward passes to compute the standard deviation, where M is the number of stochastic passes per instance.

A.4 Validation loss-based Strategy

Glister [46] is formulated as a bi-level optimization:

$$S^* = \overbrace{\arg min_{S\subseteq D,|S|\leq k}\ L_V(\underbrace{\arg min_\theta L_T(\theta, S)}_{\text{Inner-loop}},\ D_{val})}^{\text{Outer-loop}} \tag{6}$$

The outer-loop aims to select a subset S from the full set D such that a model trained on S achieves the lowest loss L_V on the validation set D_{val}. In the inner-loop, the model parameters θ are optimized by minimizing the training loss L_T on the selected subset S. The computational complexity for selecting a subset S in a single round is $O(SN(F+B))$. However, in practice, `Glister` employs an active learning strategy to iteratively solve the selection problem, resulting in a time complexity of $O(NB)$ per selection.

DVRL [101] explicitly models the data selection process as a reinforcement learning problem. DVRL leverages the validation loss as the reward, which serves as a signal to learn the likelihood of each data being selected. RL-based strategies generally incur a time complexity of $O(SN(F+B))$ per training iteration.

A.5 Influence Function Strategy

Strategies such as `CG-influence` [47] and `Arnoldi` [71] both approximate the H_θ^{-1} through iterative methods. `Arnoldi` iteratively constructs a Krylov subspace for efficient approximation, while `CG-influence` applies the conjugate gradient to compute it. In practice, both methods have a total complexity of $O(N|D_{val}MP|)$ for computing the influence scores between all training and validation instances, where M is the number of iteration steps. Typically, only a subset of validation instances is referenced rather than the entire validation set, to reduce computational cost. `TracIn` [68], on the other hand, estimates the impact by accumulating the dot products of gradients between each training instance and each validation instance across multiple training checkpoints: $\text{TracIn}(z_i, z_{\text{val}}) = \sum_{t\in\mathcal{T}}\eta_t\left\langle\nabla_\theta\mathcal{L}(z_{\text{val}}, f_{\theta_t}), \nabla_\theta\mathcal{L}(z_i, f_{\theta_t})\right\rangle$. To compute influence scores for all training-validation pairs, the overall complexity is $O(N|D_{val}|TP)$, where T is the number of checkpoints used.

A.6 Shapley Value Strategy

Common approaches include Monte Carlo sampling (e.g., `TMC-shapley` [23]), which reduces the number of required model evaluations by randomly sampling M permutations of the dataset and computing marginal contributions along each permutation, thereby lowering the overall complexity to $O(MN(F+B))$, where F and B denote the time for one forward and one backward model pass, respectively.

`CS-shapley` are designed to provide a more fine-grained and accurate assessment of data value in classification tasks by decoupling intra-class and inter-class contributions when estimating Shapley Values. Therefore, `CS-shapley` achieves superior performance to conventional Shapley Value estimators in classification tasks. In terms of computational complexity, `CS-shapley` typically estimates intra-class Shapley Values by running Monte Carlo sampling (e.g., TMCS) within each class. If there are C classes and the average class size is N_c, the complexity for intra-class estimation is $O(C, M, N_c, (F+B)) = O(MN(F+B))$.

B Appendix: Evaluation Details

B.1 Evaluation Setting Details

B.1.1 Evaluation Datasets Details Following [23], we split each out-of-distribution dataset (`MNIST`→`MNISTM`, `MNISTM`→`MNIST`, `House`, and `HR`) into three parts: (1) the training set, drawn from the source domain (i.e., `MNIST` in `MNIST`→`MNISTM`, `MNISTM` in `MNISTM`→`MNIST`, the Sales, International, Purchasing, Marketing, and Technology departments in `HR`, and house prices from Bangalore, Chennai, Delhi, Kolkata, and Hyderabad in `House`); (2) a held-out set from the target domain, consisting of 20% of randomly selected data instances (e.g., `MNISTM` in `MNIST`→`MNISTM`, `MNIST` in `MNISTM`→`MNIST`, the Finance department in `HR`, and Mumbai in `House`); (3) the final test set, comprising the remaining 80% of data instances from the target domain.

B.1.2 Evaluation Hyperparameters Details The hyperparameters used for different datasets are listed below. They are obtained by performing automated hyperparameter optimization using Optuna to identify the best overall configuration for each dataset. All methods are then trained on the same dataset and model using this unified set of hyperparameters to ensure a fair comparison. For all experiments, we employ the SGD optimizer (with momentum and weight decay) and the CosineAnnealingLR scheduler during training.

B.1.3 LLMs Trainging Setting Details We adopted LoRA [34], a parameter-efficient fine-tuning strategy, to conduct all experiments. The training employed a learning rate schedule that combined linear warm-up with cosine decay, peaking at a learning rate of 2×10^{-5}. We used a batch size of 128 and trained the models for 4 epochs on each selected dataset.

Method	10%	30%	50%	70%
Random	30.1±0.2	56.8±0.2	69.5±0.4	75.7±0.3
K-means	37.5±0.2	66.3±0.2	69.8±0.2	77.5±0.2
Herding	40.7±0.5	65.7±0.5	76.2±0.8	79.3±0.2
Kcenter	51.7±0.3	76.1±0.4	78.3±0.3	80.5±0.2
Craig	68.3±0.7	78.9±0.5	79.9±0.3	82.6±0.2
GradMatch	77.4±1.1	79.7±0.6	80.3±0.4	83.1±0.3
Least-CF	40.3±2.2	64.6±2.1	75.6±2.4	77.4±1.5
DFAL	49.6±1.2	78.0±1.3	79.5±1.2	81.1±1.2
Mean-STD	44.6±0.3	66.9±0.3	73.0±0.3	76.0±0.3
Glister	67.7±0.7	73.1±0.4	79.2±0.5	81.9±0.5
DVRL	64.2±0.5	68.2±0.4	77.5±0.3	81.7±0.3
TracIn	63.4±0.6	72.4±0.5	79.1±0.5	81.8±0.4
CG-influence	58.4±0.3	76.1±0.5	79.5±0.5	82.5±0.6
Arnoldi	61.4±0.3	76.3±0.3	79.7±0.3	82.5±0.2
TMC-shapley	62.4±0.2	74.6±0.2	79.8±0.2	82.3±0.2
CS-shapley	63.1±0.2	76.4±0.2	79.8±0.2	82.5±0.2
Full	**83.3**			

Table 8: Performance of different data selection methods on CIFAR-100 using ResNet50.

Dataset	Model	Learning Rate	Momentum	Weight Decay
Adult	MLP	0.08932	0.82756	1.50×10^{-4}
CIFAR-100	ResNet18	0.05480	0.78840	4.37×10^{-6}
CIFAR-100	ResNet50	0.07141	0.67961	1.26×10^{-6}
CIFAR-100	ViT	0.03127	0.76237	1.06×10^{-6}
CoverType	MLP	0.00004	0.81380	1.01×10^{-3}
HR	MLP	0.00349	0.98742	1.35×10^{-4}
HR	TabNet	0.00292	0.78025	3.95×10^{-4}
HR	TabTransformer	0.01563	0.78121	1.35×10^{-4}
IMDB	MLP	0.00442	0.89273	2.24×10^{-5}
IMDB-Large	MLP	0.00701	0.61016	3.52×10^{-3}
News	MLP	0.01432	0.84604	5.47×10^{-6}
Tiny-ImageNet	ResNet50	0.02604	0.74296	1.29×10^{-4}
MNIST→MNISTM	ResNet50	0.07345	0.97754	2.68×10^{-5}
MNISTM→MNIST	ResNet50	0.01873	0.98678	1.27×10^{-6}
Bike	MLP	0.00088	0.89725	1.81×10^{-3}
House	MLP	0.00011	0.78032	4.3×10^{-4}

Table 7: Evaluation hyperparameters for each dataset and model.

To balance the number of training steps across datasets of varying sizes, we initially explored adjusting the number of epochs for smaller datasets. However, this adjustment did not lead to performance gains, so we standardized the training procedure to a fixed number of epochs for all cases.

The LoRA configuration involved a rank of 128, an α parameter of 512, and a dropout probability of 0.1. LoRA adapters were applied to all attention matrices. Under this setup, the LLAMA-2-7B model had 135 million trainable parameters, amounting to 1.95% of the full model. The MISTRAL-7B variant contained 109 million trainable parameters, or 1.48% of the complete model.

Each experiment was repeated three times using different random seeds. For random sampling methods, this meant drawing three distinct subsets from the training data. In contrast, our proposed method (LESS) involved warm-up training on various data splits, followed by subset selection from the resulting warm-up models for each trial. Optimization seeds were kept consistent throughout all experiments to ensure comparability.

C Appendix: Evaluation Results

C.1 Deep Learning Evaluation Results

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0	410	1	1364	3	2250	4	3067
K-means	22	415	69	1314	114	2169	156	2962
Herding	21	407	71	1340	117	2226	161	3050
Kcenter	22	411	71	1354	119	2250	162	3085
Craig	112	450	323	1293	535	2142	723	2889
GradMatch	114	457	338	1351	541	2162	734	2937
Least-CF	26	487	75	1428	124	2357	166	3163
DFAL	48	913	96	1816	141	2676	181	3440
Mean-STD	76	1446	120	2290	168	3207	206	3924
Glister	40	772	92	1753	148	2812	193	3659
DVRL	43	832	93	1780	154	2924	206	3921
TracIn	44	837	95	1807	157	2985	212	4025
CG-influence	1204	439	1818	1363	1357	2212	1451	3062
Arnoldi	1214	433	1927	1345	1365	2287	1462	3158
TMC-shapley	2953	468	3499	1332	3127	2084	3649	2433
CS-shapley	2990	493	3543	1362	3223	2148	3680	2453
Full	0	4541	0	4541	0	4541	0	4541

Table 9: Latency (seconds) of different data selection methods on CIFAR-100, split into selection and training time.

Method	0.1	0.3	0.5	0.7
Random	81.2	81.8	83.1	83.7
K-means	82.4±1.2	84.0±2.2	84.4±1.4	84.4±0.7
Herding	83.0±0.3	84.2±0.3	84.2±0.4	84.4±0.2
Kcenter	81.2±0.3	83.7±0.3	84.0±0.4	84.8±0.2
Craig	83.6±0.6	84.3±0.6	85.1±0.7	85.5±0.4
GradMatch	83.7±0.6	84.7±0.6	85.3±0.7	85.6±0.3
Least-CF	81.5±1.3	83.0±1.3	84.0±1.3	84.5±1.2
DFAL	79.7±0.6	84.0±0.5	84.1±0.4	84.5±0.5
Mean-STD	83.2±0.4	84.1±0.5	84.3±0.6	84.7±0.3
Glister	82.6±0.3	83.5±0.3	84.8±0.3	85.2±0.3
DVRL	83.3±0.3	84.2±0.3	84.2±0.3	84.6±0.3
TracIn	83.2±0.3	83.6±0.4	84.1±0.3	85.1±0.3
CG-influence	82.3±0.2	83.2±0.3	84.6±0.5	85.1±0.3
Arnoldi	82.1±0.3	83.4±0.3	84.9±0.5	85.2±0.3
TMC-shapley	81.7±0.2	82.7±0.2	84.2±0.1	85.0±0.2
CS-shapley	83.2±0.2	83.8±0.2	85.0±0.1	85.1±0.3
Full	**85.6**			

Table 10: Performance of different data selection methods on Adult using MLP.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.01	10.99	0.03	26.97	0.05	52.95	0.08	80.92
K-means	1.1	20.9	1.6	30.4	2.85	54.15	4.2	79.8
Herding	1.05	19.95	1.95	37.05	2.95	56.05	4.3	81.7
Kcenter	1.2	22.8	2.0	38.0	3.1	58.9	4.35	82.65
Craig	9.2	36.8	11.4	45.6	13.8	55.2	17.8	71.2
GradMatch	9.6	38.4	11.6	46.4	14.6	58.4	18.8	75.2
Least-CF	2.05	38.95	2.4	45.6	3.4	64.6	4.6	87.4
DFAL	2.85	54.15	3.7	70.3	5.25	99.75	6.4	121.6
Mean-STD	3.65	69.35	4.85	92.15	6.05	114.95	7.5	142.5
Glister	3.9	74.1	4.75	90.25	5.85	111.15	6.85	130.15
DVRL	8.15	154.85	12.55	238.45	17.05	323.95	21.6	410.4
TracIn	6.75	128.25	9.85	187.15	12.75	242.25	16.4	311.6
CG-influence	154.4	33.6	223.45	45.55	255.9	62.1	299.35	87.65
Arnoldi	149.35	37.65	212.7	41.3	256.15	66.85	309.55	81.45
TMC-shapley	204.6	36.4	266.6	44.4	322.2	64.8	355.2	86.8
CS-shapley	207	38	288.6	42.4	341.2	60.8	380.2	86.8
Full	0	124	0	124	0	124	0	124

Table 11: Latency (seconds) of different data selection methods on Adult, split into selection and training time.

Method	0.1	0.3	0.5	0.7
Random	34.3±1.3	48.9±2.2	52.1±2.6	53.9±1.5
K-means	27.5±0.3	43.8±0.3	50.7±0.2	52.8±0.2
Herding	27.6±0.4	38.0±0.6	47.8±0.7	52.1±0.4
Kcenter	36.6±0.3	48.9±0.3	51.3±0.4	53.2±0.3
Craig	42.2±0.6	49.7±0.6	53.2±0.5	54.5±0.5
GradMatch	44.2±0.4	50.3±0.5	53.5±0.6	54.6±0.5
Least-CF	22.7±2.2	40.2±2.4	50.2±1.8	52.8±0.4
DFAL	36.6±0.4	47.3±0.6	52.1±0.4	53.4±0.4
Mean-STD	37.8±0.3	50.0±0.3	52.2±0.3	53.6±0.3
Glister	35.1±0.3	38.1±0.5	42.8±0.3	52.2±0.3
DVRL	38.1±0.2	48.7±0.3	52.2±0.3	53.7±0.3
TracIn	37.0±0.4	49.8±0.3	52.2±0.4	54.4±0.3
CG-influence	39.0±0.3	49.5±0.2	52.1±0.4	53.9±0.3
Arnoldi	37.5±0.3	41.3±0.3	50.9±0.3	53.7±0.3
TMC-shapley	36.9±0.3	41.3±0.3	50.2±0.3	54.2±0.3
CS-shapley	36.6±0.4	48.5±0.3	51.6±0.3	54.3±0.3
Full	**56.7**			

Table 12: Performance of different data selection methods on News using MLP.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.01	13.99	0.03	31.97	0.06	58.94	0.07	73.93
K-means	0.8	15.2	1.8	34.2	3.25	61.75	3.9	74.1
Herding	1.05	19.95	1.95	37.05	3.35	63.65	4.05	76.95
Kcenter	1.0	19.0	2.0	39.0	3.3	62.7	4.1	77.9
Craig	8.0	32.0	10.2	40.8	15.2	60.8	18.4	73.6
GradMatch	8.4	33.6	10.6	42.4	15.6	62.4	19.0	76.0
Least-CF	1.7	32.3	2.45	46.55	3.65	69.35	4.45	84.55
DFAL	3.2	60.8	4.1	77.9	5.65	107.35	6.2	117.8
Mean-STD	3.85	73.15	4.7	89.3	6.35	120.65	7.7	146.3
Glister	3.45	65.55	4.35	82.65	5.15	97.85	6.3	118.7
DVRL	7.15	135.85	11.85	225.15	15.8	300.2	20.4	387.6
TracIn	17.7	336.3	20.65	392.35	24.35	462.65	27.55	523.45
CG-influence	1191.2	32.8	1212.8	43.2	1235.0	62.0	1256.65	76.35
Arnoldi	1201.9	36.1	1223.4	44.6	1235.0	62.0	1277.85	79.15
TMC-shapley	1454.6	36.4	1471.4	47.6	1493	62	1513	75
CS-shapley	1458.2	38.8	1492.2	44.8	1520.4	63.6	1566.4	77.6
Full	0	112	0	112	0	112	0	112

Table 13: Latency (seconds) of different data selection methods on News.

Method	0.1	0.3	0.5	0.7
Random	34.8±2.3	38.5±2.3	40.7±1.1	43.4±0.4
K-means	36.6±0.3	39.8±0.5	42.5±0.3	44.1±0.4
Herding	33.6±0.2	37.8±0.3	40.5±0.3	43.7±0.4
Kcenter	35.7±0.3	41.7±0.3	43.5±0.3	43.7±0.3
Craig	38.6±0.5	42.5±0.7	45.1±0.4	45.6±0.1
GradMatch	38.8±0.3	42.8±0.3	45.5±0.1	45.6±0.1
Least-CF	37.3±1.3	39.7±2.1	42.1±2.4	45.1±1.1
DFAL	39.4±0.5	43.2±0.4	45.1±0.6	47.6±0.5
Mean-STD	39.8±0.3	43.4±0.3	45.3±0.6	47.4±0.5
Glister	43.5±0.3	44.5±0.6	45.3±0.5	47.6±0.4
DVRL	43.5±0.3	44.7±0.3	45.6±0.4	48.7±0.4
TracIn	43.6±0.3	45.3±0.3	45.7±0.3	48.8±0.3
CG-influence	43.8±0.3	45.4±0.3	46.2±0.3	50.4±0.3
Arnoldi	43.9±0.4	45.5±0.4	46.4±0.5	50.3±0.4
TMC-shapley	44.1±0.3	45.6±0.4	46.8±0.4	50.8±0.5
CS-shapley	44.9±0.3	45.6±0.3	47.3±0.4	51.2±0.5
Full	**45.6**			

Table 14: Performance of different data selection methods on MNIST→MNISTM using ResNet50.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.2	194.8	0.6	597.4	1.0	1011.0	1.4	1394.6
K-means	11.2	211.8	32.4	615.6	53.1	1007.9	72.4	1375.6
Herding	10.1	191.9	31.3	594.7	59.1	1121.9	74.5	1414.5
Kcenter	10.6	200.4	31.6	600.4	60.6	1150.4	74.9	1422.1
Craig	66.8	267.2	153.8	615.2	287.4	1149.6	323.6	1294.4
GradMatch	71.4	285.6	157.6	630.4	292.4	1169.6	326.6	1306.4
Least-CF	11.7	222.3	34.5	654.5	66.3	1259.7	77.1	1463.9
DFAL	23.2	440.8	41.5	787.5	76.2	1446.8	92.2	1751.8
Mean-STD	27.9	530.1	46.9	890.1	79.4	1507.6	94.9	1804.1
Glister	23.2	440.8	41.5	787.5	76.2	1446.8	92.2	1751.8
DVRL	77.6	698.4	116.2	1045.8	146.9	1322.1	171.2	1540.8
TracIn	588.4	882.6	636.4	954.6	694.8	1042.2	766.4	1149.6
CG-influence	1636.8	205.2	1459.6	689.4	1480.8	1171.2	1543.2	1314.8
Arnoldi	1595.6	218.4	1464.8	697.2	1486.0	1104.0	1490.0	1335.0
TMC-shapley	2121.4	247.6	2038.6	692.4	2071.2	1180.8	2115.8	1277.2
CS-shapley	2912.8	275.2	3014.6	676.4	3175.8	1117.2	2996.4	1297.6
Full	0	2055	0	2055	0	2055	0	2055

Table 15: Latency (seconds) of different data selection methods on MNIST→MNISTM.

Method	0.1	0.3	0.5	0.7
Random	1.101±0.008	0.987±0.007	0.895±0.006	0.882±0.007
K-means	0.937±0.003	0.922±0.003	0.897±0.004	0.888±0.004
Herding	0.935±0.003	0.921±0.002	0.891±0.003	0.882±0.002
Kcenter	0.934±0.003	0.919±0.003	0.888±0.003	0.873±0.003
Craig	0.884±0.005	0.873±0.005	0.866±0.004	0.862±0.003
GradMatch	0.883±0.007	0.872±0.004	0.864±0.003	0.862±0.003
Least-CF	-			
DFAL	-			
Mean-STD	0.881±0.003	0.871±0.003	0.862±0.003	0.857±0.003
Glister	0.875±0.002	0.867±0.006	0.856±0.005	0.846±0.003
DVRL	0.873±0.006	0.863±0.006	0.852±0.005	0.844±0.005
TracIn	0.873±0.005	0.853±0.006	0.847±0.004	0.843±0.003
CG-influence	0.869±0.005	0.854±0.005	0.846±0.007	0.843±0.005
Arnoldi	0.869±0.003	0.853±0.005	0.844±0.007	0.841±0.005
TMC-shapley	0.868±0.003	0.843±0.005	0.836±0.004	0.821±0.003
CS-shapley	0.867±0.006	0.842±0.006	0.835±0.005	0.819±0.005
Full	**0.862**			

Table 16: Performance of different data selection methods on House using MLP.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.2	18.8	0.6	62.4	1.1	110.9	1.6	155.4
K-means	0.4	6.6	0.4	16.6	0.5	49.5	0.6	70.4
Herding	0.4	6.6	0.7	13.3	3.0	57.0	4.2	79.8
Kcenter	0.4	7.6	0.6	10.4	1.0	19.0	4.3	81.7
Craig	8.4	47.6	11.7	66.3	17.1	96.9	20.0	113.0
GradMatch	8.0	45.0	12.0	68.0	15.8	89.2	19.2	108.8
Least-CF	-							
DFAL	-							
Mean-STD	3.3	61.7	3.9	73.1	6.1	114.9	7.2	136.8
Glister	26.2	104.8	56.0	224.0	73.6	294.4	116.8	467.2
DVRL	58.8	235.2	87.6	350.4	112.2	448.8	144.0	576.0
TracIn	221.4	516.6	246.3	574.7	303.6	708.4	379.5	885.5
CG-influence	1878.4	49.6	1921.9	51.1	1923.0	67.0	1924.3	86.7
Arnoldi	1931.9	41.1	1968.4	59.6	1926.0	64.0	2041.0	84.0
TMC-shapley	2307.5	47.5	2358.5	58.5	2433.5	63.5	2460.5	80.5
CS-shapley	2426.0	46.0	2471.5	51.5	2486.0	66.0	2524.5	84.5
Full	0	230	0	230	0	230	0	230

Table 17: Latency (seconds) of different data selection methods on House.

Method	0.1	0.3	0.5	0.7
random	0.00 ± 0.01	0.13 ± 0.01	0.17 ± 0.01	0.21 ± 0.01
kmeans	0.26 ± 0.01	0.60 ± 0.01	0.63 ± 0.01	0.59 ± 0.01
herding	0.35 ± 0.01	0.51 ± 0.01	0.64 ± 0.01	0.70 ± 0.01
kcentergreedy	0.36 ± 0.01	0.52 ± 0.01	0.66 ± 0.01	0.71 ± 0.01
craig	0.44 ± 0.01	0.65 ± 0.01	0.66 ± 0.01	0.72 ± 0.01
gradmatch	0.43 ± 0.01	0.67 ± 0.01	0.67 ± 0.01	0.72 ± 0.01
Least-CF	0.29 ± 0.05	0.56 ± 0.01	0.58 ± 0.02	0.64 ± 0.02
DFAL	0.53 ± 0.01	0.53 ± 0.01	0.64 ± 0.01	0.71 ± 0.01
mean_std	0.58 ± 0.02	0.61 ± 0.01	0.65 ± 0.04	0.69 ± 0.01
Glister	0.26 ± 0.01	0.57 ± 0.05	0.69 ± 0.01	0.71 ± 0.00
dvrl	0.52 ± 0.03	0.53 ± 0.03	0.65 ± 0.03	0.68 ± 0.03
Arnoldi	0.54 ± 0.03	0.55 ± 0.03	0.66 ± 0.03	0.68 ± 0.03
TracIn	0.50 ± 0.01	0.52 ± 0.01	0.61 ± 0.01	0.69 ± 0.01
CG-influence	0.51 ± 0.01	0.53 ± 0.01	0.64 ± 0.01	0.70 ± 0.01
TMCS	0.52 ± 0.01	0.54 ± 0.01	0.61 ± 0.01	0.70 ± 0.01
cs_shapley	0.53 ± 0.01	0.53 ± 0.01	0.64 ± 0.01	0.71 ± 0.01
Full	**0.728**			

Table 18: Performance of different data selection methods on Tiny-ImageNet using ResNet50

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.6	526.6	0.68	1765.2	0.62	2030.61	0.61	3210.97
K-means	83.22	1425.17	92.42	1528.40	93.40	2329.22	95.27	2764.86
Herding	129.51	1624.36	243.09	2613.38	345.60	3619.09	403.03	3312.53
Kcenter	71.82	876.00	96.04	1840.94	101.53	3809.08	113.88	4580.65
Craig	243.50	752.44	249.97	1666.98	259.70	2388.86	271.65	3286.69
GradMatch	243.50	752.44	249.97	1666.98	259.70	2388.86	271.65	3286.69
Least-CF	31.76	766.92	38.69	1530.75	41.11	2546.35	48.14	1852.73
DFAL	31.99	578.07	37.40	874.16	42.14	1137.52	46.78	1663.52
Mean-STD	2719.81	461.85	2714.38	907.42	2730.73	1130.50	3049.52	1704.20
DVRL	9827.29	979.68	25765.10	927.87	42054.31	2758.56	55985.75	3517.87
TracIn	266.36	433.25	269.04	2151.68	273.73	2898.34	280.80	5156.46
Arnoldi	1864.55	706.96	1979.03	1791.80	1740.10	2220.07	1851.72	3667.00
TMC-shapley	2216.05	708.90	2220.73	1631.53	2227.31	2277.73	2233.87	3727.58
CS-shapley	2242.64	730.74	2251.21	1980.98	2257.83	2895.40	2260.57	3666.25
Full	0	14673	0	14673	0	14673	0	14673

Table 19: Latency (seconds) of different data selection methods on Tiny-ImageNet.

Method	0.1	0.3	0.5	0.7
Random	74.8 ± 0.4	75.5 ± 0.2	76.1 ± 0.3	76.8 ± 0.2
K-means	62.2 ± 0.3	63.5 ± 0.2	63.7 ± 0.4	70.3 ± 0.4
Herding	61.1 ± 0.2	62.1 ± 0.4	69.1 ± 0.3	71.1 ± 0.5
Kcenter	64.1 ± 0.3	73.2 ± 0.2	75.5 ± 0.4	75.8 ± 0.3
Craig	73.0 ± 0.8	76.7 ± 0.5	77.3 ± 0.6	78.3 ± 0.4
GradMatch	**77.1** ± 0.6	77.3 ± 0.5	78.2 ± 0.5	78.4 ± 0.4
Least-CF	70.9 ± 0.4	73.7 ± 0.5	74.4 ± 0.4	76.5 ± 0.5
DFAL	72.5 ± 0.3	73.0 ± 0.4	74.2 ± 0.5	76.8 ± 0.4
Mean-STD	75.9 ± 0.3	76.8 ± 0.2	77.6 ± 0.4	78.8 ± 0.3
DVRL	**77.1** ± 1.0	78.4 ± 0.8	79.9 ± 0.6	80.8 ± 0.4
Glister	71.2 ± 0.5	75.0 ± 0.6	76.8 ± 0.7	79.3 ± 0.7
TracIn	74.1 ± 0.6	77.4 ± 0.5	78.7 ± 0.5	79.6 ± 0.4
Arnoldi	**77.1** ± 0.9	78.4 ± 0.7	78.7 ± 0.6	82.6 ± 0.7
CG-influence	**77.1** ± 0.9	78.4 ± 0.7	79.7 ± 0.7	82.4 ± 0.7
TMC-shapley	76.1 ± 1.0	**79.7** ± 1.1	**81.7** ± 1.1	**82.9** ± 1.2
CS-shapley	**77.1** ± 1.0	78.8 ± 0.9	80.9 ± 0.9	**82.9** ± 1.0
Full	78.5			

Table 20: Performance of different data selection methods on HR using MLP.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.01	10.96	0.03	47.40	0.05	82.62	0.09	100.15
K-means	1.05	60.38	1.52	71.90	2.97	95.90	4.33	116.13
Herding	1.08	61.39	1.87	76.34	2.69	98.04	4.41	113.22
Kcenter	1.17	62.23	2.07	78.48	3.36	99.65	4.27	111.60
Craig	8.86	76.44	12.03	86.92	14.12	94.61	16.81	111.90
GradMatch	9.21	79.62	11.28	85.07	15.61	99.77	18.51	116.10
Least-CF	2.18	78.56	2.55	86.14	3.55	105.71	4.50	126.59
DFAL	2.93	93.99	3.56	111.08	5.19	137.85	6.78	161.01
Mean-STD	3.78	109.11	4.69	131.77	5.82	156.01	7.43	183.27
Glister	4.07	115.13	4.70	129.06	5.61	149.74	7.02	171.62
DVRL	8.57	153.42	13.09	238.97	17.49	325.72	20.34	416.91
TracIn	6.24	165.92	10.32	229.32	12.47	278.96	15.97	350.40
CG-influence	170.11	60.97	232.55	73.41	256.80	98.42	310.25	118.01
Arnoldi	160.02	65.30	223.17	70.44	265.38	98.56	318.68	112.95
TMC-shapley	273.71	67.89	234.56	75.71	424.01	95.72	526.80	119.90
CS-shapley	266.14	69.88	256.91	73.72	439.65	91.24	539.13	117.88
Full	0	177	0	177	0	177	0	177

Table 21: Latency (seconds) of different data selection methods on HR.

Method	0.1	0.3	0.5	0.7
Random	81.2	81.8	83.1	83.7
K-means	82.3±1.2	84.2±2.2	84.4±1.4	84.6±0.7
Herding	83.2±0.3	84.2±0.3	84.2±0.4	84.6±0.2
Kcenter	81.4±0.3	83.7±0.3	84.3±0.4	84.8±0.2
Craig	83.6±0.6	84.8±0.6	85.2±0.6	85.5±0.3
GradMatch	83.4±0.6	84.8±0.6	85.6±0.2	85.6±0.2
Least-CF	81.6±1.3	83.1±1.3	84.2±1.3	84.6±1.2
DFAL	79.9±0.6	84.2±0.5	84.3±0.4	84.7±0.5
Mean-STD	83.4±0.4	84.3±0.5	84.5±0.6	84.8±0.3
Glister	82.8±0.3	83.6±0.3	84.9±0.3	85.3±0.3
DVRL	83.5±0.3	84.1±0.3	84.1±0.3	84.5±0.3
TracIn	83.2±0.3	83.4±0.4	84.3±0.3	85.3±0.3
CG-influence	82.3±0.2	83.3±0.3	84.8±0.5	85.2±0.3
Arnoldi	82.3±0.3	83.6±0.3	85.3±0.5	85.4±0.3
TMC-shapley	81.9±0.2	82.7±0.2	84.4±0.1	85.2±0.2
CS-shapley	83.4±0.2	84.1±0.2	85.2±0.1	85.3±0.3
Full	**85.8**			

Table 22: Performance of different data selection methods on IMDB using MLP.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.0	33.71	0.0	26.42	0.0	26.30	0.0	25.69
K-means	0.78	9.43	0.77	12.60	0.76	16.45	0.77	23.10
Kcenter	2.70	5.86	5.43	9.45	8.17	12.17	10.45	17.29
Herding	3.01	10.18	6.15	16.92	9.27	22.45	11.85	24.54
Craig	14.37	15.91	15.36	22.64	16.43	27.12	16.49	27.76
GradMatch	3.47	12.21	3.61	14.31	3.56	19.08	3.49	27.65
Least-CF	0.0	19.66	0.0	22.20	0.0	30.42	0.0	31.20
DFAL	22.81	15.08	23.88	13.95	23.92	20.05	23.88	23.47
Mean-STD	15.58	12.21	15.25	15.23	15.30	14.97	15.73	19.55
Glister	91.27	3.54	89.53	7.80	91.03	11.07	90.73	33.76
DVRL	57.57	10.31	128.93	10.89	199.65	20.40	263.14	39.74
TracIn	2.82	21.09	2.94	26.18	2.94	26.68	2.89	38.78
CG-influence	20.69	12.62	20.57	11.98	19.98	14.78	20.46	34.63
Arnoldi	119.45	13.85	118.54	18.50	119.27	20.88	118.07	24.84
TMC-shapley	307.05	17.37	303.63	16.62	303.06	26.69	303.63	27.89
CS-shapley	2935.90	13.25	2927.03	17.45	2917.43	31.99	2917.16	38.54
Full	0	97	0	97	0	97	0	97

Table 23: Latency (seconds) of different data selection methods on IMDB.

Method	0.1	0.3	0.5	0.7
Random	66.9±0.3	66.5±0.3	66.7±0.2	66.5±0.2
K-means	58.7±1.2	64.3±0.8	64.7±0.7	65.5±0.5
Herding	47.0±2.0	64.3±1.0	65.0±0.6	66.4±0.4
Kcenter	65.7±0.5	66.1±0.4	65.6±0.3	65.4±0.2
Craig	66.6±0.9	66.6±0.5	66.7±0.9	66.4±0.8
GradMatch	66.6±0.3	66.7±0.3	66.9±0.2	67.1±0.2
Least-CF	57.0±1.4	63.1±0.8	63.6±0.5	65.9±0.4
DFAL	56.4±1.1	57.4±0.9	64.0±0.5	65.8±0.4
Mean-STD	66.5±0.2	66.4±0.2	66.8±0.1	66.8±0.1
Glister	61.1±0.7	63.2±0.6	65.5±0.4	66.2±0.3
DVRL	61.7±0.7	64.6±0.6	65.4±0.4	66.3±0.3
TracIn	62.9±0.5	65.2±0.5	65.6±0.3	66.4±0.2
Arnoldi	50.1±3.2	49.1±3.7	48.7±4.0	57.9±2.6
CG-influence	63.4±0.7	64.0±0.7	64.8±0.6	66.2±0.4
TMC-shapley	66.5±0.2	66.7±0.2	66.9±0.1	66.6±0.1
CS-shapley	66.3±0.2	66.5±0.2	66.6±0.1	66.8±0.1
Full	**69.6**			

Table 24: F1 performance of different data selection methods on IMDB-Large using MLP.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.99	1952.34	1.19	2465.04	1.55	3148.35	1.74	4065.27
K-means	58.67	2255.02	65.29	13436.04	67.78	6593.27	70.96	7990.12
Herding	53790.76	1429.95	139579.40	2531.30	217313.51	4372.23	268814.85	8304.99
Kcenter	6942.70	4378.63	20740.00	2127.79	34532.28	3176.19	48463.65	6696.62
GradMatch	3175.40	4579.79	3134.24	5801.60	3095.53	4056.10	3093.95	7012.76
Least-CF	130.26	1896.88	132.96	2898.59	142.81	3936.65	148.12	4937.14
DFAL	186.69	2979.02	197.94	4642.67	202.20	6242.24	216.43	7922.82
Mean-STD	7360.06	2284.67	7362.90	5101.37	7332.18	8771.53	7374.60	9328.89
Glister	7252.26	1963.52	7332.33	3921.45	7372.08	6523.36	7772.08	6823.36
DVRL	27372.32	3644.90	65547.71	2484.78	101823.10	3453.25	137210.51	5374.81
TracIn	470.90	2669.73	478.26	4387.57	497.87	7191.54	567.57	9878.14
Arnoldi	120300.69	2999.87	124845.86	6137.86	115537.31	10756.07	121461.90	11950.93
CG-influence	2841.95	2798.79	8590.09	4464.18	14366.82	7922.00	22104.44	8082.11
TMC-shapley	9230.22	2736.72	9243.97	5948.94	9258.70	9508.39	9263.03	9554.60
CS-shapley	5303.47	2731.27	3752.67	4063.42	4415.02	5920.28	4818.28	7368.66
Full	0	12412	0	12412	0	12412	0	12412

Table 25: Latency (seconds) of different data selection methods on IMDB-Large, split into selection and training time.

Method	0.1	0.3	0.5	0.7
Random	32.0±0.0	32.0±0.0	32.0±0.0	32.0±0.0
K-means	32.0±0.0	36.3±2.0	32.0±0.1	36.0±2.1
Herding	32.0±0.0	32.0±0.0	32.0±0.0	32.0±0.0
Kcenter	32.0±0.0	32.0±0.0	32.0±0.0	35.4±1.8
Craig	32.2±0.2	34.1±1.1	34.5±2.5	36.5±0.2
GradMatch	35.7±1.9	32.0±0.0	34.1±0.3	36.0±0.5
Least-CF	33.4±0.8	32.0±0.0	32.0±0.0	32.0±0.0
DFAL	32.0±0.0	32.0±0.0	32.0±0.0	32.0±0.0
Mean-STD	32.0±0.0	32.0±0.0	32.0±0.0	32.0±0.0
Glister	32.0±0.0	33.8±1.0	32.0±0.1	36.7±2.4
DVRL	32.0±0.0	32.0±0.0	32.6±0.4	32.0±0.0
TracIn	35.4±1.7	32.0±0.0	35.9±1.8	35.5±1.8
Arnoldi	32.0±0.0	32.0±0.0	32.0±0.0	35.2±1.7
CG-influence	32.0±0.0	32.0±0.0	32.0±0.0	32.0±0.0
TMC-shapley	32.0±0.0	32.0±0.0	32.0±0.0	32.0±0.0
CS-shapley	32.0±0.0	32.0±0.0	32.0±0.0	33.4±0.8
Full	**36.7**			

Table 26: F1 scores of different data selection methods on CoverType.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	1.49	61.98	1.48	81.02	1.49	96.82	1.53	112.55
K-means	6.30	116.40	7.12	184.98	7.97	286.80	8.83	578.26
Herding	93.98	100.11	328.75	199.71	576.58	297.12	826.06	377.90
Kcenter	34.99	99.67	90.07	183.16	144.98	258.94	204.72	633.85
Craig	87.70	77.14	89.00	124.62	88.02	171.20	90.73	217.13
GradMatch	158.77	78.74	158.82	126.01	162.79	179.18	161.44	216.32
Least-CF	9.55	99.02	10.36	126.69	11.02	170.30	12.10	221.52
DFAL	922.49	82.89	849.11	121.21	694.96	164.07	816.87	216.28
Mean-STD	338.97	90.11	332.79	142.04	333.70	166.48	337.19	215.47
Glister	113.52	117.18	147.25	212.89	168.72	374.34	195.86	396.59
DVRL	1496.93	72.31	3748.43	102.73	6194.68	156.36	7874.10	468.08
TracIn	423.51	112.30	624.10	189.23	826.68	636.48	991.49	683.47
CG-influence	2844.03	94.18	2922.14	116.63	2972.38	176.42	3018.19	401.45
Arnoldi	2832.84	130.98	2798.67	223.64	2788.62	232.67	2818.30	415.48
TMC-shapley	9383.34	121.79	11454.31	190.63	12150.98	245.73	12461.57	465.75
CS-shapley	9476.34	133.71	11327.44	191.82	12150.81	248.36	12241.35	467.55
Full	0	655	0	655	0	655	0	655

Table 27: Latency (seconds) of different data selection methods on CoverType.

Method	0.1	0.3	0.5	0.7
Random	82.4±1.2	88.1±1.0	2.3±0.2	82.5±1.2
K-means	97.5±0.3	98.5±0.2	98.7±0.2	96.9±0.6
Herding	84.0±1.0	95.4±0.6	94.9±0.5	98.0±0.3
Kcenter	93.8±0.6	96.0±0.5	95.3±0.4	96.2±0.4
Craig	95.9±0.5	98.4±0.2	98.1±0.3	98.2±0.2
GradMatch	97.9±0.2	98.5±0.2	98.7±0.2	98.6±0.2
Least-CF	97.9±0.2	98.4±0.2	94.2±0.9	99.0±0.2
DFAL	97.0±0.5	98.3±0.3	99.1±0.1	98.7±0.2
Mean-STD	97.9±0.2	98.4±0.2	94.2±0.9	99.0±0.2
Glister	97.8±0.3	98.4±0.2	98.6±0.2	98.8±0.2
DVRL	97.9±0.2	98.4±0.2	98.6±0.2	98.8±0.2
TracIn	97.9±0.2	98.4±0.2	98.6±0.2	99.0±0.1
CG-influence	97.9±0.2	98.4±0.2	98.6±0.2	99.0±0.1
Arnoldi	97.9±0.2	98.4±0.2	98.6±0.2	99.1±0.1
TMC-shapley	98.0±0.2	98.5±0.2	98.6±0.2	99.1±0.1
CS-shapley	98.0±0.2	98.5±0.2	98.6±0.2	99.2±0.1
Full	**98.82**			

Table 28: F1 score of different data selection methods on MNISTM→MNIST.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.55	146.45	0.57	1301.59	0.58	601.65	0.57	1305.86
K-means	38.46	538.85	41.08	824.07	43.74	1036.99	44.60	3217.78
Herding	56.31	213.56	100.56	570.21	133.41	1031.19	159.19	1078.54
Kcenter	37.68	338.90	47.66	1058.74	55.89	1636.99	64.68	1987.11
Craig	555.70	713.35	571.33	1543.83	586.47	2967.64	612.78	4483.43
	60.22	180.15	110.12	510.25	156.43	1135.10	178.99	2001.60
Mean-STD	1603.85	224.46	1615.26	459.93	1607.15	2084.75	1613.15	1219.91
DFAL	7452.12	686.86	6687.71	2496.01	6010.91	4032.89	5297.02	3108.66
Glister	105.11	155.91	142.31	373.54	193.99	999.65	220.22	1786.44
DVRL	5365.82	116.87	13936.60	344.40	22229.15	660.94	30715.70	2524.42
TracIn	153.05	94.94	157.73	212.37	163.88	612.88	168.50	890.63
CG-influence	1099.70	315.11	1112.33	598.14	1129.10	1104.37	1145.55	1912.80
Arnoldi	1600.32	383.47	1629.99	511.01	1652.10	1150.23	1699.77	2131.64
TMC-shapley	2110.12	488.10	2141.23	595.60	2176.22	1184.45	2203.44	2102.77
CS-shapley	2250.10	393.11	2295.12	511.31	2344.11	1001.23	2399.00	1950.99
Full	0	2825.37	0	2825.37	0	2825.37	0	2825.37

Table 29: Latency (seconds) of different data selection methods on MNISTM→MNIST.

Method	0.1	0.3	0.5	0.7
Random	1.0150±0.020	1.0120±0.021	1.0118±0.023	0.9850±0.025
K-means	0.9760±0.011	1.0457±0.015	1.0491±0.012	1.0254±0.010
Herding	0.9978±0.008	0.9930±0.009	1.0088±0.010	1.0097±0.009
Kcenter	1.0050±0.010	1.0090±0.011	1.0083±0.012	1.0064±0.010
Craig	0.9978±0.007	0.9930±0.008	0.9867±0.009	0.9826±0.008
GradMatch	0.9958±0.008	0.9893±0.009	0.9863±0.003	0.9824±0.001
Least-CF	1.0458±0.019	1.0030±0.017	1.0247±0.018	0.9959±0.019
DFAL	1.0016±0.014	1.0367±0.014	1.0284±0.015	1.0078±0.013
Glister	1.0397±0.010	1.0225±0.011	1.0222±0.010	1.0162±0.009
DVRL	1.0218±0.013	1.0141±0.012	1.0120±0.012	1.0069±0.010
TracIn	1.0040±0.010	1.0020±0.010	1.0000±0.009	0.9980±0.009
Arnoldi	1.0240±0.012	1.0120±0.011	1.0080±0.010	0.9960±0.009
CG-influence	1.0010±0.008	0.9980±0.008	0.9940±0.008	0.9910±0.007
TMC-shapley	1.0000±0.006	0.9970±0.006	0.9940±0.006	0.9890±0.006
CS-shapley	0.9970±0.005	0.9960±0.005	0.9890±0.005	0.9840±0.005
Full	**0.9812**			

Table 30: MSE of different data selection methods on Bike.

Method	0.1		0.3		0.5		0.7	
	Sel.	Train	Sel.	Train	Sel.	Train	Sel.	Train
Random	0.2	7.4	0.6	26.4	1.1	49.7	1.6	66.7
K-means	0.4	9.6	0.4	27.6	0.5	48.5	0.6	68.4
Herding	0.4	8.9	0.7	30.3	3.0	47.0	4.2	69.8
Kcenter	0.4	7.6	0.6	30.4	1.0	49.0	4.3	71.7
Craig	8.4	44.7	11.7	64.2	16.2	93.1	17.8	99.1
GradMatch	8.0	43.5	12.0	68.1	15.8	89.6	19.6	98.8
Least-CF	-							
DFAL	-							
Mean-STD	3.4	62.4	4.1	72.9	5.9	113.9	8.1	126.6
Glister	27.1	103.1	56.4	227.3	73.7	292.1	111.1	469.7
DVRL	58.7	233.6	88.1	354.3	113.1	449.7	144.0	576.9
TracIn	220.8	522.5	237.9	577.7	322.7	705.8	382.7	889.2
CG-influence	576.3	9.7	593.2	30.8	611.4	47.1	624.3	67.7
Arnoldi	583.8	8.8	597.9	29.6	632.3	48.8	641.2	69.8
TMC-shapley	1177.5	9.5	1208.5	29.5	1233.5	47.5	1260.5	70.5
CS-shapley	1236.0	9.0	1261.5	30.5	1286.0	48.0	1324.5	69.5
Full	0	94	0	94	0	94	0	94

Table 31: Latency (seconds) of different data selection methods on Bike.

C.2 LLMs Domain Evaluation Results

The tables below report the zero-shot, 2-shot, and 5-shot performance of LLaMA2-7B and Mistral-7B on ARC_challenge, OpenBookQA, GSM8K, CommonsenseQA evaluation benchmarks. For the SelectIT method, we follow the settings described in its original paper [57] and conduct evaluations at the token, sentence, and model levels. Consistent with the analysis in the main text, LESS continues to show the strongest in-domain advantage, with Superfilter performing second best.

Method	0.01	0.1	0.3	0.5
Random	0.4741 ± 0.0146	0.4829 ± 0.0146	0.5019 ± 0.0146	0.5102 ± 0.0146
DSIR	0.4863 ± 0.0146	0.5034 ± 0.0146	0.5026 ± 0.0146	0.4989 ± 0.0146
RDS	0.5082 ± 0.0146	0.5122 ± 0.0146	0.5256 ± 0.0146	0.5060 ± 0.0146
BM25	0.4983 ± 0.0146	0.4932 ± 0.0146	0.5026 ± 0.0146	0.4932 ± 0.0146
LESS	0.5117 ± 0.0146	0.5384 ± 0.0146	0.5578 ± 0.0146	0.5427 ± 0.0146
SHED	0.4983 ± 0.0146	0.4957 ± 0.0146	0.4932 ± 0.0146	0.4932 ± 0.0146
Superfilter	0.5129 ± 0.0146	0.5223 ± 0.0146	0.5266 ± 0.0146	0.5302 ± 0.0146
TAGCOS	0.4846 ± 0.0146	0.4863 ± 0.0146	0.4949 ± 0.0146	0.5009 ± 0.0146
PPL	0.5085 ± 0.0146	0.5077 ± 0.0146	0.5115 ± 0.0146	0.5074 ± 0.0146
NLL	0.4975 ± 0.0146	0.5051 ± 0.0146	0.5051 ± 0.0146	0.5094 ± 0.0146
SelectIT (token)	0.4889 ± 0.0146	0.4829 ± 0.0146	0.4932 ± 0.0146	0.5017 ± 0.0146
SelectIT (sentence)	0.4846±0.0146	0.4881±0.0146	0.4991±0.0146	0.5068 ± 0.0146
SelectIT (model)	0.4787 ± 0.0146	0.4898±0.0146	0.4940±0.0146	0.5094 ± 0.0146
Full	0.5166 ± 0.0146			

Table 32: Llama2-7B Performance of different data selection methods on the ARC_Challenge dataset (0-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is ARC_Challenge.

Method	0.01	0.1	0.3	0.5
Random	0.5003 ± 0.0146	0.5139 ± 0.0146	0.5282 ± 0.0146	0.5267 ± 0.0146
DSIR	0.5101 ± 0.0146	0.5135 ± 0.0146	0.5175 ± 0.0146	0.5184 ± 0.0146
RDS	0.5235 ± 0.0146	0.5280 ± 0.0145	0.5486 ± 0.0145	0.5401 ± 0.0146
BM25	0.5258 ± 0.0146	0.5289 ± 0.0146	0.5392 ± 0.0146	0.5350 ± 0.0146
LESS	0.5392 ± 0.0146	0.5597 ± 0.0146	0.5657 ± 0.0146	0.5580 ± 0.0146
SHED	0.5103 ± 0.0146	0.5256 ± 0.0146	0.5384 ± 0.0146	0.5410 ± 0.0146
Superfilter	0.5282 ± 0.0146	0.5323 ± 0.0146	0.5373 ± 0.0146	0.5397 ± 0.0146
TAGCOS	0.5107 ± 0.0146	0.5145 ± 0.0146	0.5182 ± 0.0146	0.5137 ± 0.0146
PPL	0.5230 ± 0.0146	0.5255 ± 0.0146	0.5324 ± 0.0146	0.5290 ± 0.0146
NLL	0.5210 ± 0.0146	0.5265 ± 0.0146	0.5290 ± 0.0146	0.5341 ± 0.0146
SelectIT (token)	0.5273 ± 0.0146	0.5265 ± 0.0146	0.5299 ± 0.0146	0.5288 ± 0.0146
SelectIT (sentence)	0.5327±0.0146	0.5358±0.0146	0.5299±0.0146	0.5256±0.0146
SelectIT (model)	0.5299±0.0146	0.5273±0.0146	0.5265 ± 0.0146	0.5273±0.0146
Full	0.5390 ± 0.0146			

Table 33: Llama2-7B Performance and latency of different data selection methods on the ARC_Challenge dataset (2-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is ARC_Challenge.

Method	0.01	0.1	0.3	0.5
Random	0.5238 ± 0.0146	0.5265 ± 0.0146	0.5375 ± 0.0146	0.5316 ± 0.0146
DSIR	0.5512 ± 0.0145	0.5333 ± 0.0146	0.5563 ± 0.0145	0.5333 ± 0.0146
RDS	0.5529 ± 0.0145	0.5580 ± 0.0146	0.5486 ± 0.0145	0.5486 ± 0.0145
BM25	0.5392 ± 0.0146	0.5256 ± 0.0146	0.5367 ± 0.0146	0.5384 ± 0.0146
LESS	0.5572 ± 0.0146	0.5631 ± 0.0146	0.5646 ± 0.0146	0.5683 ± 0.0146
SHED	0.5255 ± 0.0146	0.5299 ± 0.0146	0.5444 ± 0.0146	0.5435 ± 0.0146
Superfilter	0.5450 ± 0.0146	0.5452 ± 0.0146	0.5487 ± 0.0146	0.5499 ± 0.0146
TAGCOS	0.5333 ± 0.0146	0.5239 ± 0.0146	0.5273 ± 0.0146	0.5299 ± 0.0146
PPL	0.5478 ± 0.0146	0.5365 ± 0.0146	0.5455 ± 0.0146	0.5482 ± 0.0146
NLL	0.5427 ± 0.0146	0.5357 ± 0.0146	0.5410 ± 0.0146	0.5444 ±0.0146
SelectIT (token)	0.5384 ± 0.0146	0.5333 ± 0.0146	0.5367 ± 0.0146	0.5230 ± 0.0146
SelectIT (sentence)	0.5374 ± 0.0146	0.5343 ± 0.0146	0.5357 ± 0.0146	0.5330 ± 0.0146
SelectIT (model)	0.5341 ± 0.0146	0.5324 ± 0.0146	0.5341 ± 0.0146	0.5340 ± 0.0146
Full	0.5550 ± 0.0146			

Table 34: Llama2-7B Performance and latency of different data selection methods on the ARC_Challenge dataset (5-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is ARC_Challenge.

Method	0.01	0.1	0.3	0.5
Random	0.1248 ± 0.0097	0.1566 ± 0.0105	0.1994 ± 0.0110	0.2237 ± 0.0115
DSIR	0.1327 ± 0.0093	0.1698 ± 0.0103	0.2077 ± 0.0112	0.2168 ± 0.0114
RDS	0.1031 ± 0.0084	0.1092 ± 0.0086	0.1478 ± 0.0098	0.1645 ± 0.0102
BM25	0.1342 ± 0.0094	0.1964 ± 0.0109	0.2350 ± 0.0117	0.2509 ± 0.0119
LESS	0.1571 ± 0.0098	0.2270 ± 0.0112	0.2654 ± 0.0115	0.2650 ± 0.0117
SHED	0.1296 ± 0.0093	0.1562 ± 0.0100	0.1964 ± 0.0109	0.2100 ± 0.0112
Superfilter	0.1527 ± 0.0093	0.2062 ± 0.0100	0.2442 ± 0.0094	0.2500 ± 0.0112
TAGCOS	0.1425 ± 0.0096	0.1873 ± 0.0107	0.2305 ± 0.0116	0.2229 ± 0.0115
PPL	0.1516 ± 0.0099	0.1994 ± 0.0110	0.2206 ± 0.0114	0.2403 ± 0.0118
NLL	0.1433 ± 0.0097	0.1804 ± 0.0106	0.2221 ± 0.0114	0.2367 ± 0.0115
SelectIT (token)	0.1622 ± 0.0102	0.2090 ± 0.0116	0.2428 ± 0.0124	0.2637 ± 0.0123
SelectIT (sentence)	0.1630 ± 0.0102	0.2138 ± 0.0113	0.2469 ± 0.0122	0.2611 ± 0.0125
SelectIT (model)	0.1698 ± 0.0103	0.2200 ± 0.0121	0.2511 ± 0.0125	0.2608 ± 0.0121
Full	0.2745 ± 0.0123			

Table 39: Llama2-7B Performance and latency of different data selection methods on the GSM8K dataset (2-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is GSM8K.

Method	0.01	0.1	0.3	0.5
Random	0.4111 ± 0.0141	0.4742 ± 0.0143	0.5348 ± 0.0143	0.5558 ± 0.0141
DSIR	0.4406 ± 0.0142	0.4038 ± 0.0140	0.5078 ± 0.0143	0.5369 ± 0.0142
RDS	0.4398 ± 0.0142	0.5225 ± 0.0143	0.5766 ± 0.0141	0.5505 ± 0.0141
BM25	0.4414 ± 0.0142	0.5577 ± 0.0142	0.6028 ± 0.0140	0.6241 ± 0.0139
LESS	0.5161 ± 0.0141	0.5652 ± 0.0141	0.6380 ± 0.0138	0.6691 ± 0.0135
SHED	0.4120 ± 0.0141	0.4939 ± 0.0143	0.5340 ± 0.0143	0.5446 ± 0.0143
Superfilter	0.5011 ± 0.0138	0.5328 ± 0.0138	0.5755 ± 0.0138	0.6096 ± 0.0141
TAGCOS	0.4665 ± 0.0143	0.4963 ± 0.0143	0.4947 ± 0.0143	0.5586 ± 0.0142
PPL	0.5070 ± 0.0143	0.5171 ± 0.0143	0.5643 ± 0.0142	0.5921 ± 0.0141
NLL	0.4939 ± 0.0143	0.5201 ± 0.0143	0.5684 ± 0.0142	0.6061 ± 0.0140
SelectIT (token)	0.4520 ± 0.0138	0.4988 ± 0.0132	0.5725 ± 0.0142	0.5897 ± 0.0141
SelectIT (sentence)	0.4594 ± 0.0138	0.5140 ± 0.0136	0.5807 ± 0.0141	0.6003 ± 0.0140
SelectIT (model)	0.4537 ± 0.0143	0.5627 ± 0.0142	0.5561 ± 0.0142	0.6061 ± 0.0140
Full	0.5938 ± 0.0141			

Table 35: Llama2-7B Performance and latency of different data selection methods on the CommonsenseQA dataset (0-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is CommonsenseQA.

Method	0.01	0.1	0.3	0.5
Random	0.4930 ± 0.0143	0.5987 ± 0.0140	0.6372 ± 0.0138	0.6716 ± 0.0134
DSIR	0.4750 ± 0.0143	0.4816 ± 0.0143	0.6200 ± 0.0139	0.6609 ± 0.0136
RDS	0.5430 ± 0.0143	0.5283 ± 0.0143	0.5782 ± 0.0141	0.6413 ± 0.0137
BM25	0.5635 ± 0.0142	0.6462 ± 0.0137	0.6724 ± 0.0134	0.6814 ± 0.0133
LESS	0.5792 ± 0.0143	0.6694 ± 0.0143	0.6732 ± 0.0134	0.6921 ± 0.0132
SHED	0.5217 ± 0.0143	0.5700 ± 0.0142	0.6241 ± 0.0139	0.6224 ± 0.0139
Superfilter	0.5437 ± 0.0143	0.6015 ± 0.0143	0.6577 ± 0.0143	0.6838 ± 0.0143
TAGCOS	0.5152 ± 0.0143	0.5938 ± 0.0141	0.6233 ± 0.0139	0.6511 ± 0.0136
PPL	0.5319 ± 0.0143	0.5935 ± 0.0142	0.6503 ± 0.0137	0.6527 ± 0.0136
NLL	0.5315 ± 0.0143	0.5949 ± 0.0142	0.6454 ± 0.0137	0.6708 ± 0.0135
SelectIT (token)	0.4988 ± 0.0143	0.5839 ± 0.0141	0.6183 ± 0.0139	0.6683 ± 0.0135
SelectIT (sentence)	0.5037 ± 0.0143	0.5823±0.0141	0.6454 ± 0.0137	0.6773 ± 0.0134
SelectIT (model)	0.4799 ± 0.0143	0.5790 ± 0.0141	0.6339 ± 0.0138	0.6691 ± 0.0135
Full	0.6929 ± 0.0132			

Table 36: Llama2-7B Performance and latency of different data selection methods on the CommonsenseQA dataset (2-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is CommonsenseQA.

Method	0.01	0.1	0.3	0.5
Random	0.5536 ± 0.0142	0.6192 ± 0.0139	0.6485 ± 0.0137	0.6822 ± 0.0133
DSIR	0.5561 ± 0.0142	0.5809 ± 0.0143	0.6347 ± 0.0138	0.6544 ± 0.0136
RDS	0.5864 ± 0.0141	0.5618 ± 0.0142	0.5921 ± 0.0141	0.6593 ± 0.0136
BM25	0.6003 ± 0.0140	0.6618 ± 0.0135	0.6763 ± 0.0133	0.6821 ± 0.0132
LESS	0.5917 ± 0.0142	0.6763 ± 0.0143	0.6922 ± 0.0133	0.7142 ± 0.0129
SHED	0.5864 ± 0.0141	0.6183 ± 0.0139	0.6519 ± 0.0136	0.6413 ± 0.0137
Superfilter	0.5845 ± 0.0142	0.6549 ± 0.0142	0.6727 ± 0.0142	0.6948 ± 0.0143
TAGCOS	0.5545 ± 0.0142	0.6110 ± 0.0140	0.6495 ± 0.0137	0.6824 ± 0.0134
PPL	0.5717 ± 0.0142	0.6061 ± 0.0140	0.6626 ± 0.0135	0.6809 ± 0.0136
NLL	0.5848 ± 0.0141	0.5987 ± 0.0140	0.6527 ± 0.0136	0.6834 ± 0.0135
SelectIT (token)	0.5684 ± 0.0142	0.6265 ± 0.0138	0.6326 ± 0.0137	0.6754 ± 0.0134
SelectIT (sentence)	0.5784 ± 0.0142	0.6255 ± 0.0138	0.6396 ± 0.0137	0.6724 ± 0.0134
SelectIT (model)	0.5643 ± 0.0143	0.6190 ± 0.0140	0.6356 ± 0.0137	0.6734 ± 0.0134
Full	0.7043 ± 0.0131			

Table 37: Llama2-7B Performance and latency of different data selection methods on the CommonsenseQA dataset (5-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is CommonsenseQA.

Method	0.01	0.1	0.3	0.5
Random	0.0735 ± 0.0072	0.0864 ± 0.0077	0.0804 ± 0.0075	0.0993 ± 0.0082
DSIR	0.0902 ± 0.0079	0.0963 ± 0.0081	0.1084 ± 0.0086	0.1615 ± 0.0101
RDS	0.0576 ± 0.0064	0.0546 ± 0.0063	0.0478 ± 0.0059	0.0591 ± 0.0065
BM25	0.0493 ± 0.0060	0.0963 ± 0.0081	0.1418 ± 0.0096	0.1289 ± 0.0092
LESS	0.1170 ± 0.0082	0.1342 ± 0.0094	0.1510 ± 0.0096	0.1524 ± 0.0099
SHED	0.0728 ± 0.0072	0.0819 ± 0.0076	0.1160 ± 0.0088	0.0963 ± 0.0081
Superfilter	0.0938 ± 0.0062	0.1087 ± 0.0078	0.1233 ± 0.0080	0.1119 ± 0.0076
TAGCOS	0.0576 ± 0.0064	0.0993 ± 0.0082	0.1090 ± 0.0089	0.1084 ± 0.0086
PPL	0.0902 ± 0.0079	0.1031 ± 0.0084	0.1130 ± 0.0087	0.1135 ± 0.0072
NLL	0.0826 ± 0.0076	0.1043 ± 0.0072	0.1137 ± 0.0087	0.1196 ± 0.0075
SelectIT (token)	0.0896 ± 0.0075	0.1221 ± 0.0090	0.1489 ± 0.0099	0.1342 ± 0.0094
SelectIT (sentence)	0.0940 ± 0.0080	0.1303 ± 0.0096	0.1476 ± 0.0103	0.1448 ± 0.0097
SelectIT (model)	0.1145 ± 0.0088	0.1333 ± 0.0097	0.1454 ± 0.0010	0.1334 ± 0.0076
Full	0.1168 ± 0.0088			

Table 38: Llama2-7B Performance and latency of different data selection methods on the GSM8K dataset (0-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is GSM8K.

Method	0.01	0.1	0.3	0.5
Random	0.1327 ± 0.0093	0.1673 ± 0.0107	0.2097 ± 0.0116	0.2449 ± 0.0118
DSIR	0.1660 ± 0.0102	0.1873 ± 0.0107	0.2153 ± 0.0113	0.2434 ± 0.0118
RDS	0.1519 ± 0.0093	0.1989 ± 0.0092	0.2074 ± 0.0105	0.2489 ± 0.0106
BM25	0.1425 ± 0.0096	0.2108 ± 0.0112	0.2547 ± 0.0120	0.2669 ± 0.0122
LESS	0.1655 ± 0.0096	0.2370 ± 0.0112	0.2697 ± 0.0115	0.2788 ± 0.0117
SHED	0.1463 ± 0.0097	0.1994 ± 0.0110	0.2077 ± 0.0112	0.2335 ± 0.0117
Superfilter	0.1640 ± 0.0097	0.2210 ± 0.0096	0.2547 ± 0.0100	0.2571 ± 0.0098
TAGCOS	0.1516 ± 0.0099	0.2062 ± 0.0111	0.2282 ± 0.0116	0.2502 ± 0.0119
PPL	0.1645 ± 0.0102	0.2191 ± 0.0114	0.2547 ± 0.0120	0.2500 ± 0.0121
NLL	0.1683 ± 0.0103	0.2033 ± 0.0109	0.2585 ± 0.0121	0.2494 ± 0.0119
SelectIT (token)	0.1577 ± 0.0100	0.2343 ± 0.0117	0.2343 ± 0.0117	0.2443 ± 0.0124
SelectIT (sentence)	0.1577 ± 0.0100	0.2343 ± 0.0117	0.2343 ± 0.0117	0.2443 ± 0.0124
SelectIT (model)	0.1857 ± 0.0107	0.2631 ± 0.0121	0.2343 ± 0.0117	0.2443 ± 0.0124
Full	0.2790 ± 0.0124			

Table 40: Llama2-7B Performance and latency of different data selection methods on the GSM8K dataset (5-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is GSM8K.

Method	0.01	0.1	0.3	0.5
Random	0.4180 ± 0.0223	0.4440 ± 0.0222	0.4400 ± 0.0222	0.4660 ± 0.0223
DSIR	0.4500 ± 0.0223	0.4480 ± 0.0223	0.4540 ± 0.0223	0.4540 ± 0.0223
RDS	0.4420 ± 0.0223	0.4480 ± 0.0223	0.4460 ± 0.0223	0.4500 ± 0.0223
BM25	0.4580 ± 0.0223	0.4540 ± 0.0223	0.4440 ± 0.0222	0.4440 ± 0.0222
LESS	0.4640 ± 0.0222	0.4720 ± 0.0223	0.4810 ± 0.0223	0.4830 ± 0.0223
SHED	0.4370 ± 0.0223	0.4440 ± 0.0222	0.4520 ± 0.0223	0.4580 ± 0.0223
Superfilter	0.4610 ± 0.0223	0.4540 ± 0.0223	0.4620 ± 0.0222	0.4600 ± 0.0223
TAGCOS	0.4540 ± 0.0223	0.4580 ± 0.0223	0.4590 ± 0.0223	0.4600 ± 0.0223
PPL	0.4540 ± 0.0223	0.4640 ± 0.0223	0.4710 ± 0.0223	0.4720 ± 0.0223
NLL	0.4540 ± 0.0223	0.4600 ± 0.0213	0.4580 ± 0.0223	0.4780 ± 0.0223
SelectIT (token)	0.4620 ± 0.0223	0.4630 ± 0.0223	0.4620 ± 0.0223	0.4640 ± 0.0223
SelectIT (sentence)	0.4560 ± 0.0223	0.4560 ± 0.0223	0.4560 ± 0.0223	0.4580 ± 0.0223
SelectIT (model)	0.4580 ± 0.0223	0.4560±0.0223	0.4600±0.0223	0.4630 ± 0.0223
Full	0.4610 ± 0.0223			

Table 41: Llama2-7B Performance and latency of different data selection methods on the `OpenBookQA` dataset (0-shot). For task-aware strategies (including `DSIR`, `RDS`, `BM25`, `LESS` and `SHED`), indicating that the validation set is `OpenBookQA`.

Method	0.01	0.1	0.3	0.5
Random	0.4580 ± 0.0224	0.4680 ± 0.0223	0.4840 ± 0.0224	0.4880 ± 0.0224
DSIR	0.4660 ± 0.0223	0.4600 ± 0.0223	0.4900 ± 0.0224	0.4900 ± 0.0224
RDS	0.4680 ± 0.0223	0.4780 ± 0.0224	0.4940 ± 0.0224	0.4820 ± 0.0224
BM25	0.4740 ± 0.0224	0.4660 ± 0.0223	0.4780 ± 0.0224	0.4880 ± 0.0224
LESS	0.4760 ± 0.0223	0.4960 ± 0.0224	0.5120 ± 0.0223	0.5100 ± 0.0223
SHED	0.4660 ± 0.0223	0.4920 ± 0.0224	0.4920 ± 0.0224	0.4780 ± 0.0224
Superfilter	0.4700 ± 0.0223	0.4710 ± 0.0223	0.4740 ± 0.0223	0.4900 ± 0.0223
TAGCOS	0.4460 ± 0.0223	0.4700 ± 0.0223	0.4580 ± 0.0223	0.4880 ± 0.0224
PPL	0.4720 ± 0.0223	0.4940 ± 0.0224	0.4760 ± 0.0224	0.4900 ± 0.0224
NLL	0.4660 ± 0.0223	0.4960 ± 0.0224	0.4860 ± 0.0224	0.4700 ± 0.0223
SelectIT (token)	0.4680 ± 0.0223	0.4900 ± 0.0224	0.4760 ± 0.0224	0.4760 ± 0.0224
SelectIT (sentence)	0.4780 ±0.0224	0.4870 ± 0.0224	0.4820 ± 0.0224	0.4660 ± 0.0223
SelectIT (model)	0.4620 ± 0.0224	0.4920 ± 0.0224	0.4760 ± 0.0224	0.4880 ± 0.0224
Full	0.5030 ± 0.0224			

Table 42: Llama2-7B Performance and latency of different data selection methods on the `OpenBookQA` dataset (2-shot). For task-aware strategies (including `DSIR`, `RDS`, `BM25`, `LESS` and `SHED`), indicating that the validation set is `OpenBookQA`.

Method	0.01	0.1	0.3	0.5
Random	0.4720 ± 0.0223	0.4860 ± 0.0224	0.4890 ± 0.0224	0.4910 ± 0.0224
DSIR	0.4700 ± 0.0223	0.4500 ± 0.0223	0.4640 ± 0.0223	0.4720 ± 0.0223
RDS	0.4740 ± 0.0224	0.4580 ± 0.0223	0.4820 ± 0.0224	0.4580 ± 0.0223
BM25	0.4880 ± 0.0224	0.4700 ± 0.0223	0.4560 ± 0.0223	0.4820 ± 0.0224
LESS	0.4910 ± 0.0223	0.4960 ± 0.0224	0.5150 ± 0.0223	0.5120 ± 0.0223
SHED	0.4720 ± 0.0224	0.4700 ± 0.0223	0.4740 ± 0.0224	0.4780 ± 0.0224
Superfilter	0.4870 ± 0.0223	0.4900 ± 0.0224	0.4980 ± 0.0223	0.4940 ± 0.0223
TAGCOS	0.4740 ± 0.0224	0.4720 ± 0.0223	0.4710 ± 0.0223	0.4740 ± 0.0223
PPL	0.4900 ± 0.0224	0.4860 ± 0.0224	0.4920 ± 0.0223	0.4980 ± 0.0223
NLL	0.4860 ± 0.0223	0.4830 ± 0.0224	0.4890 ± 0.0223	0.4910 ± 0.0223
SelectIT (token)	0.4740 ± 0.0224	0.4740 ± 0.0224	0.4760 ± 0.0224	0.4660 ± 0.0223
SelectIT (sentence)	0.4680 ± 0.0224	0.4710 ± 0.0224	0.4730 ± 0.0224	0.4610 ± 0.0223
SelectIT (model)	0.4520 ± 0.0223	0.4820 ± 0.0224	0.4730 ± 0.0224	0.4620 ± 0.0223
Full	0.5080 ± 0.0223			

Table 43: Llama2-7B Performance and latency of different data selection methods on the `OpenBookQA` dataset (5-shot). For task-aware strategies (including `DSIR`, `RDS`, `BM25`, `LESS` and `SHED`), indicating that the validation set is `OpenBookQA`.

Method	0.01	0.1	0.3	0.5
Random	0.5440 ± 0.0145	0.5414 ± 0.0145	0.5461 ± 0.0145	0.5520 ± 0.0145
DSIR	0.5506 ± 0.0145	0.5495 ± 0.0145	0.5478 ± 0.0145	0.5316 ± 0.0146
RDS	0.5506 ± 0.0145	0.5640 ± 0.0145	0.5350 ± 0.0146	0.5324 ± 0.0223
BM25	0.5600 ± 0.0145	0.5401 ± 0.0146	0.5316 ± 0.0146	0.5358 ± 0.0146
LESS	0.5572 ± 0.0145	0.5648 ± 0.0145	0.5672 ± 0.0146	0.5742 ± 0.0146
SHED	0.4820 ± 0.0224	0.4700 ± 0.0223	0.4740 ± 0.0224	0.4780 ± 0.0224
Superfilter	0.5640 ± 0.0145	0.5569 ± 0.0145	0.5669 ± 0.0145	0.5635 ± 0.0146
TAGCOS	0.5461 ± 0.0145	0.5614 ± 0.0145	0.5469 ± 0.0145	0.5452 ± 0.0146
PPL	0.5566 ± 0.0145	0.5563 ± 0.0145	0.5529 ± 0.0145	0.5674 ± 0.0145
NLL	0.5551 ± 0.0145	0.5563 ± 0.0145	0.5529 ± 0.0145	0.5674 ± 0.0145
SelectIT (token)	0.5442 ± 0.0144	0.5557 ± 0.0145	0.5512 ± 0.0145	0.5614 ± 0.0145
SelectIT (sentence)	0.5442 ± 0.0144	0.5452 ± 0.0146	0.5520 ± 0.0145	0.5648 ± 0.0145
SelectIT (model)	0.5375 ± 0.0146	0.5555 ± 0.0145	0.5452 ± 0.0146	0.5461 ± 0.0145
Full	0.5512 ± 0.0145			

Table 44: Mistral-v0.3-7B Performance and latency of different data selection methods on the `ARC_challenge` dataset (0-shot). For task-aware strategies (including `DSIR`, `RDS`, `BM25`, `LESS` and `SHED`), indicating that the validation set is `ARC_challenge`.

Method	0.01	0.1	0.3	0.5
Random	0.5712 ± 0.0143	0.5832 ± 0.0143	0.5851 ± 0.0144	0.5864 ± 0.0143
DSIR	0.5730 ± 0.0144	0.5798 ± 0.0143	0.5873 ± 0.0143	0.5819 ± 0.0144
RDS	0.5773 ± 0.0143	0.5841 ± 0.0143	0.5825 ± 0.0145	0.5879 ± 0.0144
BM25	0.5826 ± 0.0142	0.5802 ± 0.0144	0.5964 ± 0.0143	0.5785 ± 0.0144
LESS	0.5840 ± 0.0143	0.5894 ± 0.0144	0.5990 ± 0.0145	0.6183 ± 0.0145
SHED	0.4820 ± 0.0224	0.4700 ± 0.0223	0.4740 ± 0.0224	0.4780 ± 0.0224
Superfilter	0.5920 ± 0.0142	0.5964 ± 0.0143	0.5964 ± 0.0143	0.5990 ± 0.0143
TAGCOS	0.5752 ± 0.0142	0.5875 ± 0.0143	0.5964 ± 0.0143	0.6007 ± 0.0143
PPL	0.5775 ± 0.0143	0.5813 ± 0.0144	0.5896 ± 0.0144	0.5853 ± 0.0143
NLL	0.5745 ± 0.0143	0.5833 ± 0.0144	0.5894 ± 0.0144	0.5861 ± 0.0143
SelectIT (token)	0.5784 ± 0.0143	0.5875 ± 0.0143	0.5828 ± 0.0144	0.5939 ± 0.0144
SelectIT (sentence)	0.5769 ± 0.0142	0.5804 ± 0.0144	0.5930 ± 0.0144	0.5913 ± 0.0144
SelectIT (model)	0.5747 ± 0.0143	0.5828 ± 0.0144	0.5751 ± 0.0144	0.5766 ± 0.0145
Full	0.5925 ± 0.0145			

Table 45: Mistral-v0.3-7B Performance and latency of different data selection methods on the `ARC_challenge` dataset (2-shot). For task-aware strategies (including `DSIR`, `RDS`, `BM25`, `LESS` and `SHED`), indicating that the validation set is `ARC_challenge`.

Method	0.01	0.1	0.3	0.5
Random	0.5220 ± 0.0223	0.5867 ± 0.0143	0.5913 ± 0.0144	0.5969 ± 0.0142
DSIR	0.5973 ± 0.0143	0.5863 ± 0.0144	0.5973 ± 0.0143	0.5845 ± 0.0144
RDS	0.5832 ± 0.0143	0.6015 ± 0.0143	0.5870 ± 0.0144	0.5981 ± 0.0143
BM25	0.5841 ± 0.0143	0.5947 ± 0.0143	0.5922 ± 0.0144	0.5776 ± 0.0144
LESS	0.5917 ± 0.0142	0.5971 ± 0.0144	0.6044 ± 0.0144	0.6169 ± 0.0144
SHED	0.4820 ± 0.0224	0.4700 ± 0.0223	0.4740 ± 0.0224	0.4780 ± 0.0224
Superfilter	0.5995 ± 0.0142	0.4740 ± 0.0224	0.5998 ± 0.0143	0.6067 ± 0.0143
TAGCOS	0.5980 ± 0.0141	0.5986 ± 0.0142	0.5879 ± 0.0144	0.5930 ± 0.0144
PPL	0.5895 ± 0.0142	0.5964 ± 0.0143	0.5973 ± 0.0143	0.5990 ± 0.0143
NLL	0.5885 ± 0.0142	0.5964 ± 0.0143	0.5973 ± 0.0143	0.5990 ± 0.0143
SelectIT (token)	0.5818 ± 0.0142	0.5852 ± 0.0142	0.5896 ± 0.0144	0.5990 ± 0.0143
SelectIT (sentence)	0.5937 ± 0.0142	0.5941 ± 0.0143	0.6049 ± 0.0143	0.5896 ± 0.0144
SelectIT (model)	0.5822 ± 0.0144	0.5862 ± 0.0144	0.5742 ± 0.0144	0.5900 ± 0.0145
Full	0.6028 ± 0.0144			

Table 46: Mistral-v0.3-7B Performance and latency of different data selection methods on the `ARC_challenge` dataset (5-shot). For task-aware strategies (including `DSIR`, `RDS`, `BM25`, `LESS` and `SHED`), indicating that the validation set is `ARC_challenge`.

Method	0.01	0.1	0.3	0.5
Random	0.0986 ± 0.0082	0.1099 ± 0.0086	0.1290 ± 0.0089	0.1452 ± 0.0089
DSIR	0.1160 ± 0.0088	0.1418 ± 0.0096	0.1385 ± 0.0095	0.1122 ± 0.0087
RDS	0.1175 ± 0.0089	0.1107 ± 0.0086	0.1683 ± 0.0103	0.1645 ± 0.0102
BM25	0.0986 ± 0.0082	0.1304 ± 0.0093	0.1691 ± 0.0103	0.1630 ± 0.0102
LESS	0.1663 ± 0.0125	0.1783 ± 0.0103	0.1926 ± 0.0109	0.2373 ± 0.0085
SHED	0.1120 ± 0.0224	0.1200 ± 0.0223	0.1340 ± 0.0224	0.1580 ± 0.0224
Superfilter	0.1690 ± 0.0089	0.1692 ± 0.0086	0.1820 ± 0.0097	0.2031 ± 0.0087
TAGCOS	0.1463 ± 0.0097	0.1789 ± 0.0106	0.1910 ± 0.0082	0.2008 ± 0.0112
PPL	0.1660 ± 0.0102	0.1585 ± 0.0101	0.1655 ± 0.0081	0.1921 ± 0.0087
NLL	0.1656 ± 0.0102	0.1588 ± 0.0101	0.1653 ± 0.0081	0.1938 ± 0.0087
SelectIT (token)	0.1122 ± 0.0087	0.1357 ± 0.0094	0.1350 ± 0.0094	0.1486 ± 0.0098
SelectIT (sentence)	0.1152 ± 0.0088	0.1168 ± 0.0088	0.1569 ± 0.0010	0.0940 ± 0.0080
SelectIT (model)	0.1099 ± 0.0106	0.1289 ± 0.0092	0.1827 ± 0.0106	0.1812 ± 0.0086
Full	0.1687 ± 0.0078			

Table 47: Mistral-v0.3-7B Performance and latency of different data selection methods on the GSM8K dataset (0-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is GSM8K.

Method	0.01	0.1	0.3	0.5
Random	0.2957 ± 0.0126	0.3654 ± 0.0133	0.3715 ± 0.0133	0.4132 ± 0.0136
DSIR	0.2828 ± 0.0124	0.3450 ± 0.0223	0.3874 ± 0.0134	0.4003 ± 0.0135
RDS	0.2608 ± 0.0121	0.2866 ± 0.0125	0.4170 ± 0.0136	0.4109 ± 0.0136
BM25	0.3245 ± 0.0129	0.3776 ± 0.0134	0.4079 ± 0.0135	0.4079 ± 0.0135
LESS	0.3692 ± 0.0133	0.4133 ± 0.0135	0.4435 ± 0.0137	0.4593 ± 0.0136
SHED	0.3320 ± 0.0224	0.3400 ± 0.0223	0.3840 ± 0.0224	0.4080 ± 0.0224
Superfilter	0.3321 ± 0.0130	0.4040 ± 0.0224	0.4254 ± 0.0128	0.4341 ± 0.0135
TAGCOS	0.3336 ± 0.0130	0.3829 ± 0.0134	0.4064 ± 0.0135	0.4139 ± 0.0136
PPL	0.3275 ± 0.0129	0.3927 ± 0.0135	0.4299 ± 0.0136	0.4056 ± 0.0135
NLL	0.3273 ± 0.0129	0.3937 ± 0.0135	0.4284 ± 0.0136	0.4076 ± 0.0135
SelectIT (token)	0.2820 ± 0.0124	0.3707 ± 0.0133	0.4276 ± 0.0136	0.4321 ± 0.0136
SelectIT (sentence)	0.3252 ± 0.0129	0.3230 ± 0.0129	0.4094 ± 0.0135	0.4321 ± 0.0136
SelectIT (model)	0.3588 ± 0.0135	0.4031 ± 0.0138	0.4302 ± 0.0137	0.4326 ± 0.0137
Full	0.4290 ± 0.0137			

Table 48: Mistral-v0.3-7B Performance and latency of different data selection methods on the GSM8K dataset (2-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is GSM8K.

Method	0.01	0.1	0.3	0.5
Random	0.3012 ± 0.0135	0.3829 ± 0.0134	0.4139 ± 0.0136	0.4193 ± 0.0136
DSIR	0.2911 ± 0.0125	0.3571 ± 0.0132	0.4094 ± 0.0135	0.3965 ± 0.0135
RDS	0.2843 ± 0.0124	0.3078 ± 0.0127	0.4352 ± 0.0137	0.4299 ± 0.0136
BM25	0.3487 ± 0.0131	0.4170 ± 0.0136	0.4257 ± 0.0136	0.4443 ± 0.0137
LESS	0.4220 ± 0.0224	0.4200 ± 0.0223	0.4540 ± 0.0224	0.4780 ± 0.0224
SHED	0.3745 ± 0.0133	0.3867 ± 0.0134	0.4412 ± 0.0137	0.4419 ± 0.0137
Superfilter	0.4023 ± 0.0133	0.4112 ± 0.0131	0.4318 ± 0.0132	0.4638 ± 0.0136
TAGCOS	0.3313 ± 0.0130	0.3965 ± 0.0135	0.4193 ± 0.0136	0.4587 ± 0.0137
PPL	0.3533 ± 0.0132	0.4147 ± 0.0136	0.4314 ± 0.0136	0.4314 ± 0.0136
NLL	0.3533 ± 0.0132	0.4147 ± 0.0136	0.4314 ± 0.0136	0.4314 ± 0.0136
SelectIT (token)	0.3101 ± 0.0127	0.3768 ± 0.0133	0.4299 ± 0.0136	0.4663 ± 0.0137
SelectIT (sentence)	0.3313 ± 0.0130	0.3381 ± 0.0130	0.4230 ± 0.0136	0.4625 ± 0.0137
SelectIT (model)	0.4064 ± 0.0135	0.4207 ± 0.0138	0.4587 ± 0.0137	0.4678 ± 0.0137
Full	0.4723 ± 0.0136			

Table 49: Mistral-v0.3-7B Performance and latency of different data selection methods on the GSM8K dataset (5-shot). For task-aware strategies (including DSIR, RDS, BM25, LESS and SHED), indicating that the validation set is GSM8K.

C.3 LLMs General Evaluation Results

Method	[ARC](CQA)			[CQA](CQA)			[OQA](CQA)			[GSM8K](CQA)			Average		
	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3
Random	0.47	0.48	0.50	0.41	0.47	0.53	0.418	0.444	0.440	0.12	0.16	0.20	0.35	0.39	0.42
DSIR	0.48	0.48	0.49	0.44	0.40	0.51	0.446	0.445	0.449	0.14	0.17	0.21	0.38	0.37	0.41
RDS	0.47	0.49	0.48	0.44	0.52	0.58	0.439	0.443	0.435	0.10	0.11	0.13	0.36	0.39	0.41
BM25	0.46	0.48	0.50	0.44	0.56	0.60	0.452	0.449	0.451	0.13	0.15	0.20	0.37	0.41	0.44
LESS	0.47	0.49	0.49	**0.52**	**0.57**	**0.64**	0.455	0.460	0.447	0.14	0.19	0.20	0.40	0.41	0.44
SHED	0.46	0.48	0.48	0.41	0.49	0.53	0.431	0.444	0.454	0.13	0.16	0.21	0.36	0.39	0.42
Superfilter	**0.51**	**0.52**	**0.52**	0.50	**0.53**	0.58	0.461	0.454	0.462	**0.15**	**0.21**	**0.24**	**0.41**	**0.43**	**0.45**
TAGCOS	0.48	0.49	0.49	0.47	0.50	0.49	0.454	0.458	0.459	0.14	0.19	0.23	0.39	0.41	0.42
PPL	**0.51**	0.51	0.51	0.51	0.50	0.56	0.454	**0.464**	**0.471**	**0.15**	0.20	0.22	**0.41**	0.42	0.44
NLL	0.50	0.51	0.51	0.50	0.52	0.57	0.454	0.458	0.460	0.14	0.18	0.22	0.40	0.42	0.44
SelectIT	0.49	0.48	0.49	0.46	0.47	0.57	**0.462**	0.463	0.462	**0.15**	**0.21**	0.23	0.39	0.41	0.44
Full		0.52			0.59			0.46			0.27			0.46	

Table 50: Performance based on general evaluation of different data selection methods. The validation set is CommonsenseQA. The bold values indicate the best performance, and underline denotes the second results. Model-agnostic, task-aware strategies are marked in orange, model-aware, task-aware strategies in blue, and model-aware, task-agnostic strategies in green. We report 0-shot results for ARC_challenge, CommonsenseQA, and OpenBookQA, and 2-shot results for GSM8K. Each experiment is run three times, and the average is reported.

Method	[ARC](OQA)			[CQA](OQA)			[OQA](OQA)			[GSM8K](OQA)			Average		
	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3
Random	0.47	0.48	0.50	0.41	0.47	0.53	0.418	0.444	0.440	0.12	0.16	0.20	0.35	0.39	0.42
DSIR	0.47	0.48	0.50	0.41	0.41	0.47	0.450	0.448	0.454	0.14	0.18	0.19	0.37	0.38	0.40
RDS	0.46	0.48	0.50	0.39	0.48	0.55	0.442	0.448	0.450	0.10	0.11	0.13	0.35	0.38	0.41
BM25	0.47	0.46	0.49	0.42	0.48	0.53	0.458	0.454	0.444	0.13	0.14	0.20	0.37	0.38	0.42
LESS	0.48	0.48	0.51	0.44	0.47	0.55	**0.464**	**0.472**	**0.481**	0.14	0.18	0.23	0.38	0.40	0.44
SHED	0.44	0.47	0.48	0.41	0.48	0.52	0.437	0.444	0.452	0.13	0.17	0.21	0.35	0.39	0.42
Superfilter	**0.51**	**0.52**	**0.52**	0.50	**0.53**	**0.58**	0.461	0.454	0.462	**0.15**	**0.21**	**0.24**	**0.41**	**0.43**	**0.45**
TAGCOS	0.48	0.49	0.49	0.47	0.50	0.49	0.454	0.458	0.459	0.14	0.19	0.23	0.39	0.41	0.42
PPL	**0.51**	0.51	0.51	**0.51**	0.50	0.56	0.454	0.464	0.471	**0.15**	0.20	0.22	**0.41**	0.42	0.44
NLL	0.50	**0.51**	0.51	0.50	0.52	0.57	0.454	0.458	0.460	0.14	0.18	0.22	0.40	0.42	0.44
SelectIT	0.49	0.48	0.49	0.46	0.47	0.57	0.462	0.463	0.462	**0.15**	**0.21**	0.23	0.39	0.41	0.44
Full		0.52			0.59			0.46			0.27			0.46	

Table 51: Performance based on general evaluation of different data selection methods. The validation set is OpenBookQA. The bold values indicate the best performance, and underline denotes the second results. Model-agnostic, task-aware strategies are marked in orange, model-aware, task-aware strategies in blue, and model-aware, task-agnostic strategies in green. We report 0-shot results for ARC_challenge, CommonsenseQA, and OpenBookQA, and 2-shot results for GSM8K. Each experiment is run three times, and the average is reported.

Method	[ARC](GSM8K)			[CQA](GSM8K)			[OQA](GSM8K)			[GSM8K](GSM8K)			Average		
	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3	0.01	0.1	0.3
Random	0.47	0.48	0.50	0.41	0.47	0.53	0.41	0.44	0.44	0.12	0.16	0.20	0.35	0.39	0.42
DSIR	0.46	0.48	0.48	0.40	0.40	0.50	0.447	0.444	0.450	0.13	0.17	0.20	0.36	0.37	0.41
RDS	0.47	0.48	0.49	0.38	0.47	54	0.441	0.443	0.436	0.10	0.11	0.15	0.35	0.38	0.40
BM25	0.47	0.46	0.49	0.41	0.47	0.51	0.451	0.448	0.451	0.13	0.20	0.24	0.37	0.39	0.42
LESS	0.47	0.50	0.50	0.44	0.47	0.51	0.445	0.449	0.451	**0.16**	**0.23**	**0.27**	0.38	0.41	0.44
SHED	0.48	0.48	0.49	0.40	0.46	0.50	0.430	0.441	0.452	0.13	0.16	0.20	0.36	0.39	0.41
Superfilter	**0.51**	**0.52**	**0.52**	0.50	**0.53**	**0.58**	0.461	0.454	0.462	0.15	0.21	0.24	**0.41**	**0.43**	**0.45**
TAGCOS	0.48	0.49	0.49	0.47	0.50	0.49	0.454	0.458	0.459	0.14	0.19	0.23	0.39	0.41	0.42
PPL	**0.51**	0.51	0.51	**0.51**	0.50	0.56	0.454	**0.464**	**0.471**	0.15	0.20	0.22	**0.41**	0.42	0.44
NLL	0.50	0.51	0.51	0.50	0.52	0.57	0.454	0.458	0.460	0.14	0.18	0.22	0.40	0.42	0.44
SelectIT	0.49	0.48	0.49	0.46	0.47	0.57	**0.462**	0.463	0.462	0.15	0.21	0.23	0.39	0.41	0.44
Full		0.52			0.59			0.46			0.27			0.46	

Table 52: Performance based on general evaluation of different data selection methods. The validation set is GSM8K. The bold values indicate the best performance, and underline denotes the second results. Model-agnostic, task-aware strategies are marked in orange, model-aware, task-aware strategies in blue, and model-aware, task-agnostic strategies in green. We report 0-shot results for ARC_challenge, CommonsenseQA, and OpenBookQA, and 2-shot results for GSM8K. Each experiment is run three times, and the average is reported.